



FACULTAD DE INGENIERIA

Universidad de Buenos Aires

Captación del entorno y mapeo local

Informe de tesis de Ingeniería Electrónica

Autor	Tomás de Aguirre	103887	tdeaguirre@fi.uba.ar
Tutor	Ignacio Mas	206872	imas@fi.uba.ar
Co-tutor	Sergio Lew	128563	slew@fi.uba.ar

Resumen

Las personas con ceguera o baja visión enfrentan dificultades significativas al desplazarse por entornos desconocidos o dinámicos. Las herramientas de asistencia tradicionales, como el bastón blanco, presentan limitaciones en alcance y resolución espacial, y no advierten eficazmente obstáculos ubicados a la altura de la cabeza. En paralelo, disciplinas como la robótica móvil han consolidado un conjunto de tecnologías de percepción, mapeo y comunicación entre procesos que permiten a los sistemas autónomos interpretar su entorno en tiempo real, pero que rara vez se aplican al problema de la movilidad personal asistida.

El eje central de esta tesis es estudiar, evaluar y adaptar tecnologías existentes, entre ellas la percepción de profundidad por visión estéreo activa, las grillas de ocupación probabilísticas, la fusión de datos inerciales y el ecosistema ROS 2. El aporte no reside en la creación de técnicas nuevas, sino en determinar cuáles resultan más adecuadas y cómo integrarlas para construir una herramienta de asistencia funcional y de bajo costo, considerando las restricciones propias de un dispositivo portátil: cómputo embebido limitado y baja latencia. La elección del canal háptico como interfaz de salida se fundamenta en el principio de sustitución sensorial: el cerebro puede reorganizarse para interpretar estímulos táctiles como información espacial cuando la visión está reducida o ausente. Sobre esa base, el proyecto global busca traducir información espacial visual en estímulos hápticos interpretables para la navegación; se estructura en dos etapas complementarias, (i) captación del entorno y mapeo local y (ii) traducción a interfaz háptica. La presente tesis, aborda exclusivamente la primera.

En concreto, se desarrolla en ROS 2 un módulo de percepción que, a partir de la cámara Intel RealSense D435i montada sobre la cabeza del usuario y ejecutándose en una Raspberry Pi 5, genera en tiempo real una representación compacta del entorno inmediato. El desarrollo sigue tres versiones incrementales. La v0 resuelve el problema de forma directa y con baja latencia: reduce la imagen de profundidad a una grilla de 5 filas $\times$ 10 columnas, la convierte en intensidades hápticas y genera la trama UART prevista para una futura interfaz física, además de ofrecer una visualización de los resultados para depuración. Sobre esa base, la v1 mejora la interpretación de las mediciones y la señal destinada a la segunda etapa: alinea la nube con la gravedad, elimina piso y techo, y mejora la distribución espacial asociada a cada motor. Por último, la v2 amplía la protección fuera del campo de visión: usa la salida filtrada de v1 para construir un mapa local de ocupación 2D egocéntrico basado en log-odds, con trazado de rayos Bresenham, desplazamiento dinámico (*scroll*) y odometría por RGB-D e IMU, conservando información reciente de obstáculos que han dejado de estar dentro del campo de visión de la cámara. Un nodo de fusión combina esta memoria espacial con la percepción frontal instantánea de v1.

El desempeño se evalúa mediante métricas cuantitativas de error de profundidad, latencia extremo a extremo, precisión y *recall* en detección de obstáculos, estabilidad temporal del mapa local y consumo de recursos computacionales en la plataforma embebida objetivo.

Abstract

People who are blind or have low vision face significant difficulties when navigating unfamiliar or dynamic environments. Traditional assistive tools such as the white cane have limited range and spatial resolution, and fail to effectively detect obstacles at head height. Meanwhile, fields such as mobile robotics have consolidated a set of perception, mapping and inter-process communication technologies that enable autonomous systems to interpret their surroundings in real time, yet these technologies are seldom applied to the problem of personal assisted mobility.

The main focus of this thesis is to study, evaluate, and adapt existing technologies, including active stereo depth sensing, probabilistic occupancy grids, inertial data fusion, and the ROS 2 ecosystem. The contribution does not lie in the creation of novel techniques, but rather in determining which of these technologies are most suitable and how to integrate them to build a functional, low-cost assistive device, while considering the constraints of a portable system: limited embedded computing resources and low latency. The choice of the haptic channel as the output interface is grounded in the principle of sensory substitution: the brain can reorganize to interpret tactile stimuli as spatial information when vision is reduced or absent. On that basis, the broader project seeks to translate visual spatial information into haptic stimuli interpretable for navigation; it is structured in two complementary stages: (i) environment capture and local mapping, and (ii) translation into a haptic interface. This thesis, addresses exclusively the first stage.

Specifically, a perception module is developed in ROS 2 that, using an Intel RealSense D435i camera mounted on the user's head and running on a Raspberry Pi 5, generates a compact real-time representation of the immediate surroundings. The development follows three incremental versions. The v0 version addresses the problem directly and with low latency: it reduces the depth image to a grid of 5 rows by 10 columns, converts it to haptic intensities, and generates the UART frame intended for a future physical interface, while also providing a visualization of the results for debugging purposes. Building on this basis, v1 improves measurement interpretation and the signal intended for the second stage: it aligns the point cloud with gravity, removes floor and ceiling points, and improves the spatial distribution assigned to each motor. Finally, v2 extends protection beyond the field of view by using the filtered output of v1 to build and maintain a 2D egocentric local occupancy map based on log-odds, with Bresenham ray casting, dynamic scrolling and RGB-D and IMU odometry, preserving recent obstacle information even after obstacles leave the camera's field of view. A fusion node combines this spatial memory with the instantaneous frontal perception of v1.

System performance is assessed through quantitative metrics including depth error, end-to-end latency, obstacle detection precision and recall, temporal stability of the local map, and computational resource consumption on the target embedded platform.

Dedicatoria

Agradecimientos

Índice general

1. Introducción	10
1.1. Contexto y motivación	10
1.2. Planteo del problema	11
1.3. Objetivos	12
1.3.1. Objetivo del proyecto	12
1.3.2. Objetivo de la tesis	12
1.3.3. Objetivo técnico	12
1.4. Contribuciones	13
1.5. Alcance	13
1.6. Organización del documento	14
2. Estado del arte	15
2.1. Dispositivos de asistencia sensorial para personas ciegas	15
2.1.1. Criterios de adopción	15
2.1.2. Sistemas sin RGB-D (sensado puntual)	16
2.1.3. Sistemas con RGB-D (percepción densa)	16
2.1.4. Producto comercial consolidado: <i>SuperBrain 1</i>	17
2.1.5. Síntesis comparativa de los sistemas revisados	18
2.2. Mapeo local y evitación de obstáculos en robótica móvil	19
2.2.1. Mapas locales reactivos	19
2.2.2. Representaciones de ocupación probabilística	19
2.2.3. Representación 2D y compensación gravitacional	20
2.2.4. Estimación del movimiento egocéntrico	20
2.3. Limitaciones identificadas y aporte de la tesis	20
3. Marco teórico	23
3.1. Fundamentos de la versión v0: percepción directa, ROS 2 y salida háptica	24
3.1.1. Sustitución sensorial y neuroplasticidad	24
3.1.2. Percepción RGB-D y geometría de cámara	24
3.1.3. ROS 2 como infraestructura de comunicación distribuida	27
3.1.4. Representación compacta y salida embebida	29
3.2. Fundamentos de la versión v1: orientación, gravedad y suelo	33
3.2.1. Orientación y alineación gravitacional	33
3.2.2. Estimación y clasificación geométrica del suelo	36
3.3. Fundamentos de la versión v2: odometría, mapa local y conciencia espacial	41
3.3.1. Estimación de pose y coherencia temporal	41
3.3.2. Representación y actualización del mapa local	44
3.3.3. Evaluación y comunicación del riesgo contextual	48
3.3.4. Métricas de evaluación para percepción y mapeo	50

4. Arquitectura del sistema	51
4.1. Arquitectura general	51
4.2. Comparación de la topología entre versiones	51
4.2.1. Topología de v0: percepción directa	51
4.2.2. Topología de v1: filtrado geométrico	52
4.2.3. Topología de v2: memoria y riesgo contextual	52
4.3. Infraestructura del sistema	52
4.3.1. Paquetes de soporte e infraestructura	53
4.3.2. Visualización y diagnóstico	53
4.3.3. Adquisición y adaptación de tópicos	54
4.4. Nodos y flujo de mensajes por versión	55
4.4.1. Flujo de v0: profundidad y salida háptica	55
4.4.2. Flujo de v1: filtrado y codificación de obstáculos	55
4.4.3. Flujo de v2: odometría, mapeo y fusión contextual	56
5. Diseño del método (pipeline de percepción)	58
6. Implementación en ROS 2	59
7. Metodología experimental	60
8. Resultados	61
9. Discusión	62
10. Conclusiones	63
11. Trabajo futuro	64
A. Trazabilidad de nodos y código fuente	65

Índice de figuras

1.1. Visión general del sistema de asistencia sensorial desarrollado en esta tesis. El sistema que corre en la Raspberry Pi 5 genera una representación compacta del entorno destinada a una futura interfaz háptica portada por el usuario.	11
3.1. Transformaciones y reducciones de información desde la escena física hasta la intensidad háptica. Las anotaciones naranjas indican la información descartada o resumida en cada etapa.	23
3.2. Geometría estéreo rectificada. Un mismo punto se proyecta en las coordenadas horizontales u_L y u_R de los planos virtuales de imagen; su diferencia define la disparidad d	25
3.3. Capas de comunicación de ROS 2 empleadas en el sistema. Los nodos acceden a las funciones de ROS 2 mediante <code>rclcpp</code> o <code>rclpy</code> , construidas sobre la biblioteca común <code>rcl</code> . La interfaz <code>RMW</code> desacopla estas bibliotecas del middleware DDS; en este trabajo se utiliza Cyclone DDS sobre Linux.	28
3.4. Reducción espacial de una imagen de profundidad de 1280×720 píxeles a una <code>DepthGrid</code> de 5 filas por 10 columnas. La captura proviene de la ejecución de <code>v0</code>	30
3.5. Codificación geométrica de una nube de obstáculos. El ángulo lateral determina la columna, la coordenada vertical determina la fila y cada celda conserva la distancia radial mínima de los puntos asignados.	31
3.6. Conversión lineal, invertida y saturada de distancia mínima válida a intensidad lógica, distinguiendo el rango de percepción del rango que produce estimulación.	32
3.7. Ángulos de Euler en la convención óptica de la cámara. Cada rotación se representa alrededor de un eje fijo del sensor.	33
3.8. Singularidad <i>gimbal lock</i> : con $\text{pitch} = -90^\circ$ la cámara apunta verticalmente hacia abajo y los ejes de roll y yaw quedan alineados.	34
3.9. Ventajas de la representación mediante cuaterniones para orientaciones compuestas y configuraciones singulares.	35
3.10. Cámara D435i en posición horizontal. El acelerómetro mide $(0, -g, 0)$ en la convención óptica usada por el pipeline.	36
3.11. Casos límite en el cálculo de la rotación mínima entre el vector de aceleración y la vertical canónica.	37
3.12. Esquema tridimensional de submuestreo por <i>voxel grid</i> . Los puntos originales dentro de cada celda ocupada se reemplazan por un único representante ubicado en el centroide de las muestras. Elaboración propia.	38
3.13. Esquema bidimensional de RANSAC para ajuste de planos. Cada hipótesis se genera a partir de tres puntos y luego se cuentan como <i>inliers</i> las muestras cuya distancia al plano candidato es menor que la tolerancia ε . Elaboración propia.	39

3.14. Diagrama de bloques del filtro de Madgwick para una implementación IMU sin magnetómetro. La estimación propagada mediante el giróscopo se corrige por descenso de gradiente a partir de la dirección de gravedad observada por el acelerómetro, y luego se integra y normaliza. Fuente: Madgwick [36, Fig. 2]. . . .	43
3.15. Evolución del log-odds de una celda inicialmente desconocida y conversión de cada estado al valor publicado en <code>OccupancyGrid.data[i]</code> , utilizando la configuración inicial previa a la calibración.	46
3.16. Recorrido discreto de Bresenham sobre la grilla: las celdas intermedias reciben evidencia libre; ante un obstáculo el extremo es ocupado y, en caso contrario, la evidencia libre se extiende hasta el límite del rango representado.	47
3.17. Geometría empleada para seleccionar un riesgo contextual y calcular el tiempo hasta colisión. La zona frontal observable queda a cargo de la percepción frontal; la evaluación contextual considera las regiones laterales y posterior.	49
4.1. Arquitectura v0.	52
4.2. Arquitectura v1.	52
4.3. Arquitectura v2.	53
4.4. Mensajes funcionales de v0. La imagen de profundidad se reduce a <code>DepthGrid</code> , se transforma en una matriz lógica <code>HapticGrid</code> y se entrega a la interfaz de transmisión.	55
4.5. Mensajes funcionales de v1. La imagen, los intrínsecos y el acelerómetro permiten obtener una nube de obstáculos alineada y codificarla conservando el contrato <code>DepthGrid-HapticGrid</code> de v0.	56
4.6. Mensajes funcionales de v2. La ruta frontal se complementa con odometría, memoria local y riesgo contextual, que se fusiona antes de publicar la matriz háptica final. Se muestran únicamente los mensajes que intervienen en el procesamiento funcional principal.	56

Índice de tablas

2.1. Comparación de sistemas de asistencia sensorial para personas ciegas según los criterios de Dakopoulos y Bourbakis [13] y dos criterios técnicos adicionales. ✓: satisfecho; ~: parcialmente satisfecho; ×: no satisfecho. La fila «Proyecto global» considera la integración de las dos etapas del proyecto.	19
2.2. Alternativas consideradas para estimar el movimiento del mapa local y configuración adoptada con <code>rtabmap_odom</code>	21
4.1. Entradas locales y tópicos efectivos de los consumidores	54
A.1. Trazabilidad de adquisición, prefiltrado IMU y percepción geométrica.	66
A.2. Trazabilidad de mapeo local, codificación de grilla y conciencia espacial.	66
A.3. Trazabilidad de fusión háptica, salida, visualización y orquestación.	66

Capítulo 1

Introducción

1.1. Contexto y motivación

La movilidad autónoma en entornos desconocidos o dinámicos exige acceder de manera continua a información sobre obstáculos, desniveles y cambios en el espacio circundante. Para las personas con ceguera o baja visión, esta información se obtiene principalmente mediante herramientas de asistencia tradicionales que resuelven funciones diferentes. El bastón blanco aporta información por contacto sobre el espacio inmediato y el estado del suelo, pero su alcance queda limitado por la longitud y el barrido realizado por el usuario. El perro guía puede anticipar y evitar obstáculos durante la marcha, aunque no comunica una representación espacial explícita del entorno. En ambos casos, una interfaz electrónica puede aportar información complementaria sobre objetos elevados, como ramas, cables o señales colgantes, antes de que entren en contacto con el usuario.

En los últimos años han surgido dispositivos electrónicos de asistencia que incorporan sensores activos [1]-[3]. Sin embargo, gran parte de esas soluciones se basa en sensores puntuales de corto alcance, lo que restringe la cantidad y calidad de información espacial disponible para el usuario. En contraste, los sensores de profundidad por visión estéreo activa, como la familia Intel RealSense [4], permiten obtener una medición densa con información tridimensional dentro de su campo de visión. Su adecuación a una plataforma portátil y de baja latencia debe evaluarse junto con el procesamiento ejecutado sobre el hardware embebido.

Este trabajo parte del principio de neuroplasticidad sensorial, según el cual el cerebro puede reorganizarse para aprovechar de manera más eficiente los sentidos restantes cuando la visión está reducida o ausente [5]. A partir de este principio, el proyecto global busca desarrollar un sistema portátil capaz de traducir información espacial visual en estímulos hápticos para asistir la navegación.

Los trabajos académicos que combinan cámaras RGB-D con retroalimentación háptica [6], [7] muestran que el enfoque es técnicamente viable. Por otro lado, productos como el *SuperBrain 1* [8] muestran el nivel de funcionalidad que puede alcanzar un sistema comercial.

Esta tesis estudia cómo obtener una representación local compacta de obstáculos mediante una cámara RGB-D y una Raspberry Pi 5, sin pretender reproducir el alcance funcional ni el nivel de validación de un producto comercial. El Capítulo 2 analiza estas soluciones en detalle y compara el enfoque adoptado en esta tesis con ellas.

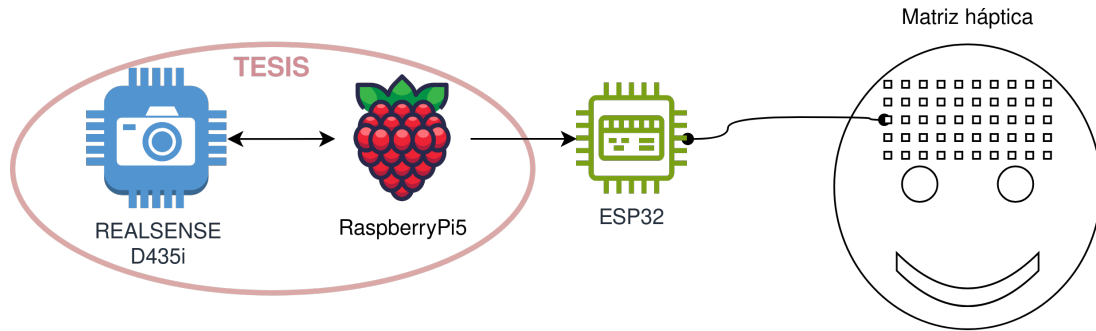


Figura 1.1: Visión general del sistema de asistencia sensorial desarrollado en esta tesis. El sistema que corre en la Raspberry Pi 5 genera una representación compacta del entorno destinada a una futura interfaz háptica portada por el usuario.

1.2. Planteo del problema

Una persona con ceguera o baja visión que se desplaza por un entorno desconocido puede beneficiarse de información actualizada sobre obstáculos ubicados desde el suelo hasta la altura de la cabeza. El bastón blanco, el perro guía y los dispositivos electrónicos ofrecen capacidades complementarias, pero no comunican el entorno con la misma cobertura, resolución ni modalidad. Los dispositivos basados en ultrasonido o sensores puntuales ofrecen información espacial acotada [2], [3]; los sistemas con cámaras de profundidad aportan mediciones más densas, a costa de una mayor demanda de procesamiento [6], [7].

El problema de ingeniería que aborda esta tesis es, por lo tanto, transformar información tridimensional del entorno en una representación compacta destinada a una interfaz háptica, utilizando una plataforma embebida y manteniendo acotados el retardo, la variación temporal y el consumo de recursos.

El proyecto global está orientado a una solución portátil. La presente tesis se limita al módulo de percepción y procesamiento, por lo que no evalúa la portabilidad del dispositivo completo, que depende además de la fuente de alimentación, la electrónica de actuación, los actuadores y el soporte corporal.

El diseño del módulo de percepción está condicionado por la plataforma embebida disponible, la necesidad de generar información reciente durante la marcha y la continuidad de la detección. En particular:

- **Plataforma embebida:** todos los componentes necesarios para generar la salida lógica se ejecutarán sobre una Raspberry Pi 5 [9], sin depender de CUDA ni de una GPU discreta. Esta condición requiere representaciones compactas y algoritmos compatibles con los recursos disponibles. La viabilidad de la implementación se evaluará mediante el uso de CPU y memoria, la temperatura de operación y la presencia de eventos de limitación térmica.
- **Desempeño temporal:** el módulo deberá producir información con un retardo y una tasa de actualización compatibles con su uso previsto durante la marcha. La evaluación abarcará desde la adquisición de la imagen hasta la publicación de la salida lógica.
- **Continuidad de la percepción:** la salida deberá evitar variaciones espurias y pérdidas intermitentes de obstáculos. En la versión con mapeo local se evaluará además el compromiso entre conservar observaciones relevantes y eliminar ocupaciones que hayan dejado de ser válidas.

- **Capacidad de detección:** el módulo deberá detectar obstáculos relevantes dentro de la región espacial y el rango de operación definidos para el sistema.

Las definiciones operativas, los criterios cuantitativos y los procedimientos de evaluación se presentan en el Capítulo 7.

1.3. Objetivos

Los objetivos del trabajo se dividen en tres niveles: el objetivo global del proyecto, el objetivo específico de la tesis, y el objetivo técnico del sistema desarrollado.

1.3.1. Objetivo del proyecto

El objetivo final del proyecto es desarrollar un sistema portátil de asistencia sensorial que permita a personas con ceguera o baja visión percibir su entorno mediante estímulos táctiles, mejorando su seguridad, autonomía y capacidad de desplazamiento independiente. El proyecto se estructura en dos etapas complementarias: (i) captación del entorno y mapeo local (objeto de esta tesis) y (ii) implementación física y validación de la interfaz háptica, desarrolladas en un trabajo separado. Como interfaz entre ambas etapas, esta tesis genera los comandos de intensidad, permite visualizarlos y deja implementada su transmisión mediante UART.

1.3.2. Objetivo de la tesis

Esta tesis aborda exclusivamente la primera etapa: captar el entorno y generar una representación local compacta a partir de percepción RGB-D. El foco se sitúa en transformar la medición cruda de profundidad en información espacial destinada a navegación asistida y caracterizar su retardo, estabilidad temporal y consumo de recursos sobre una plataforma embebida.

1.3.3. Objetivo técnico

El objetivo técnico es desarrollar un módulo que transforme mediciones de profundidad en una representación espacial compacta y en una matriz lógica de intensidades destinada a la segunda etapa del proyecto. Para materializar y evaluar ese objetivo se adoptan ROS 2 [10], una cámara Intel RealSense D435i [4] y una Raspberry Pi 5 como decisiones de implementación.

Específicamente, el módulo deberá desarrollarse en tres niveles incrementales:

- **v0 (percepción instantánea):** procesar imágenes de profundidad mediante reducción de dimensionalidad para producir una grilla estructurada de baja resolución con estadísticas por celda (distancia mínima, media y conteo de puntos válidos), y publicar dicha salida en tópicos ROS 2 estandarizados, visualizarla sin depender del hardware físico y serializarla para su transmisión mediante UART. Esta versión establece la referencia funcional directa con la cual comparar el costo de las etapas posteriores;
- **v1 (percepción filtrada):** incorporar etapas de filtrado de la medición para mejorar la relevancia de la señal destinada a navegación asistida, suprimiendo contribuciones espurias del entorno y estabilizando la representación ante variaciones de pose del portador. Su efecto sobre las activaciones falsas y la consistencia de la salida se evaluará experimentalmente;

- **v2 (mapeo local):** implementar una capa de mapeo de corto alcance basada en grillas de ocupación log-odds [11], [12], con memoria temporal de obstáculos, mecanismo de desplazamiento de ventana egocéntrica e integración de información inercial para compensar cambios de orientación durante la actualización del mapa. Su capacidad para conservar información sobre obstáculos fuera del campo frontal se evaluará en los escenarios definidos para v2.

Los parámetros numéricos y umbrales de decisión se refinarán mediante una fase de calibración y pruebas experimentales, descritas en el Capítulo 7.

1.4. Contribuciones

Los principales aportes técnicos de esta tesis son::

- una arquitectura incremental que permite comparar una transformación directa de profundidad con una interpretación geométrica filtrada y con una extensión dotada de memoria espacial, manteniendo una interfaz lógica común;
- una estrategia de filtrado geométrico alineada con la gravedad para estudiar la reducción de activaciones producidas por piso y techo sobre cómputo embebido;
- una representación compacta que separa la percepción métrica frontal del riesgo contextual y permite verificar ambas señales sin depender de la interfaz háptica física;
- un esquema de memoria local y fusión que permite mantener la advertencia cuando un obstáculo previamente observado sale del campo de visión;
- un protocolo experimental reproducible para caracterizar error, detección, latencia, estabilidad temporal y consumo de recursos.

1.5. Alcance

El alcance de esta tesis comprende:

- el diseño, implementación y validación experimental de la etapa de captación del entorno del sistema de asistencia sensorial;
- la integración del módulo desarrollado con el ecosistema ROS 2, la generación y visualización de la matriz de intensidades y su serialización para transmisión mediante UART;
- la generación de representaciones compactas de un mapa de ocupación para navegación asistida sobre hardware embebido;
- la caracterización cuantitativa del desempeño del sistema en términos de precisión, latencia y consumo de recursos.

Esta tesis incluye la generación de la matriz de intensidades hápticas, su publicación mediante mensajes ROS 2, su visualización mediante una interfaz virtual y la implementación del enlace UART previsto para integrarla con el hardware de actuación. Quedan *fuera* de su alcance el diseño y la implementación de la interfaz háptica física —microcontrolador receptor, electrónica

de potencia, actuadores y soporte corporal—, así como su caracterización y la validación con usuarios finales.

La plataforma embebida adoptada es una Raspberry Pi 5 [9]. Las pruebas experimentales caracterizarán el uso de CPU, memoria y latencia para determinar el margen operativo de la implementación sobre esa plataforma, no para decidir posteriormente su selección.

1.6. Organización del documento

La organización del documento sigue la evolución v0–v1–v2 definida en la Sección 1.3.3. Cada capítulo analiza estas versiones desde una perspectiva diferente para evitar repetir su descripción.

El Capítulo 2 revisa el estado del arte en dispositivos de asistencia sensorial con retroalimentación háptica y en técnicas de mapeo local y evitación de obstáculos en robótica móvil, y compara el enfoque adoptado con las soluciones existentes. El Capítulo 3 desarrolla los fundamentos teóricos del sistema: percepción de profundidad por estereoscopia activa, modelo de cámara *pinhole*, reducción espacial mediante partición en regiones de interés, grillas de ocupación log-odds, algoritmo de Bresenham, estimación de orientación con IMU y protocolo de comunicación serie. El Capítulo 4 describe la arquitectura modular del sistema implementada en ROS 2 y los cambios de topología introducidos en cada versión. El Capítulo 5 justifica las decisiones de procesamiento y los criterios mediante los cuales las mediciones afectan la salida. El Capítulo 6 expone la implementación concreta en C++ y Python sobre ROS 2, describiendo los nodos principales (`depth_to_matrix_node`, `local_mapper_node`, `haptic_grid_generator`, `hardware_manager`) y las interfaces definidas. El Capítulo 7 presenta la metodología experimental, los escenarios de prueba y las métricas de evaluación utilizadas: error RMS de profundidad, latencia *end-to-end*, precisión y *recall* de detección de obstáculos, estabilidad temporal por celda y consumo de recursos. El Capítulo 8 expone los resultados empíricos obtenidos y su análisis cuantitativo. Finalmente, los Capítulos 9, 10 y 11 discuten las limitaciones del enfoque, formulan las conclusiones del trabajo y proponen líneas de trabajo futuro, respectivamente.

Capítulo 2

Estado del arte

2.1. Dispositivos de asistencia sensorial para personas ciegas

Este capítulo presenta una revisión narrativa de antecedentes seleccionados por su relación con las decisiones centrales de la tesis: modalidad de sensado, forma de realimentación, resolución espacial, memoria temporal y plataforma de cómputo. La selección no pretende ser exhaustiva ni sostener afirmaciones sobre toda la bibliografía disponible; las conclusiones se limitan a los trabajos incluidos y citados en las secciones siguientes.

El principio de sustitución sensorial, cuya base experimental se describe en el Capítulo 3 (Section 3.1.1), constituye el fundamento común de los dispositivos electrónicos de asistencia a la movilidad revisados en este capítulo. Desde los primeros arreglos de 20×20 taxeles (píxeles táctiles) de Bach-y-Rita [5], la evolución técnica puede leerse como una continuidad: se parte de soluciones con sensado puntual y se avanza hacia sistemas con percepción densa mediante cámaras de profundidad, en búsqueda de mejor resolución espacial sin perder portabilidad ni baja latencia.

A partir del trabajo de Bach-y-Rita, se desarrolló una amplia variedad de dispositivos electrónicos de ayuda a la movilidad (*Electronic Travel Aids*, ETA). Dakopoulos y Bourbakis [13] presentan una revisión exhaustiva de estos sistemas, clasificándolos según el tipo de sensor (ultrasónico, infrarrojo, láser, cámara), el mecanismo de retroalimentación (auditivo, háptico, voz) y la zona del cuerpo en la que presentan la estimulación (mano, muñeca, torso, cabeza). A partir de estos antecedentes, los sistemas revisados se dividen en dos grupos: sistemas sin RGB-D, basados en sensado puntual, y sistemas con RGB-D, que incorporan percepción densa y, en algunos casos, mapeo local.

2.1.1. Criterios de adopción

Para encuadrar la evaluación comparativa, Dakopoulos y Bourbakis enumeran cuatro criterios que un dispositivo portátil de asistencia debe cumplir para resultar adoptable en la práctica: (1) *manos libres*, sin requerir el uso de las manos para un dispositivo *adicional* al bastón blanco, que se asume como herramienta base y cuyo uso no viola este criterio; (2) *oídos libres*, sin interferir con la capacidad de escucha del entorno, que es una fuente de información crítica; (3) *vestible*, para ofrecer flexibilidad y aprovechar las ventajas de las tecnologías portátiles; y (4) *simple*, de operación intuitiva y sin necesidad de entrenamiento extenso. A estos criterios se agregan dos dimensiones técnicas adicionales relevantes para el presente trabajo: *resolución espacial* del sensor y *memoria temporal* en la representación del entorno.

2.1.2. Sistemas sin RGB-D (sensado puntual)

Entre las propuestas sin uso de RGB-D incluidas en esta revisión se distinguen tres enfoques:

Arreglos vibrotáctiles corporales

Novich y Eagleman [1] estudiaron los límites perceptuales de la piel como canal de información y concluyeron que un arreglo de actuadores sobre el torso puede alcanzar tasas de transferencia del orden de bits por segundo, suficientes para codificar patrones espaciales rudimentarios. Desde el punto de vista de los criterios de adopción, el sistema satisface las condiciones de *manos libres*, *oídos libres* y de dispositivo vestible. Sin embargo, requiere movimientos corporales activos para explorar el entorno, lo que compromete el criterio de *simplicidad* durante la marcha. La resolución espacial es limitada y no se contempla memoria temporal de las observaciones.

Bastones inteligentes

Santos et al. [2] proponen un bastón instrumentado con sensores ultrasónicos que retransmite la información de proximidad mediante vibración en el mango. Al incorporar los sensores directamente en el bastón blanco, el criterio de *manos libres* se cumple, dado que no se introduce un dispositivo adicional que ocupe las manos. El sistema es simple en su operación y no interfiere con la audición. La limitación estructural más relevante es que la detección queda restringida a la zona barrida por el bastón, sin cobertura vertical, lo que resulta en baja resolución espacial efectiva. Tampoco incorpora memoria temporal de obstáculos.

Dispositivos ultrasónicos portátiles

Petsiuk y Pearce [3] describen una pulsera ultrasónica de bajo costo que emite vibraciones al detectar objetos en un cono estrecho frente al brazo. El dispositivo es vestible y no ocupa las manos, aunque la posición en la muñeca requiere orientación activa del brazo. No interfiere con la audición y su operación es simple. No obstante, la resolución espacial es muy limitada: solo se detecta la distancia al obstáculo más próximo en la dirección del brazo, sin capacidad de representar la distribución espacial del entorno, y sin ninguna forma de acumulación temporal de observaciones.

2.1.3. Sistemas con RGB-D (percepción densa)

La incorporación de sensores de profundidad por estereoscopia activa en dispositivos de asistencia representa una continuidad natural de los ETA, orientada a superar la baja resolución de los sensores puntuales sin abandonar los criterios de adopción discutidos en la Section 2.1.1. Los sistemas revisados a continuación muestran distintos niveles de madurez dentro de esa misma línea evolutiva, desde retroalimentación directa sobre el fotograma instantáneo, pasando por integración de mapeo local con fusión inercial, hasta productos comerciales que consolidan el enfoque, como se discute en la Section 2.1.4.

Percepción instantánea sin memoria temporal

Yang et al. [14] proponen un sistema portátil basado en el sensor Intel RealSense R200 que detecta el área transitable y comunica la información al usuario mediante retroalimentación sonora estereofónica. Su trabajo demuestra que la densidad espacial del sensor RGB-D se puede explotar en tiempo real sobre hardware embebido para mejorar significativamente la cobertura

de detección respecto de sistemas ultrasónicos, siendo validado con participantes invidentes en entornos interiores y exteriores.

No obstante, el trabajo de Yang et al. no incorpora información inercial para compensar la inclinación del sensor respecto al plano horizontal, ni implementa una etapa de mapeo local. La representación final se obtiene directamente del fotograma instantáneo, por lo que toda información sobre obstáculos se pierde en cuanto estos abandonan el campo de visión de la cámara. Esto impide acumular un historial de observaciones que podría ser utilizado por una lógica de evitación de colisiones para anticipar obstáculos que, aunque ya no visibles en ese instante, siguen siendo relevantes para la trayectoria del usuario.

Sistemas con mapeo local e integración inercial

Un antecedente más completo es el sistema propuesto por Lee y Medioni [6], que integra una cámara RGB-D montada en la cabeza con una IMU de nueve grados de libertad, una grilla de ocupación 2D probabilística y un chaleco de retroalimentación vibrotáctil. El sistema estima la pose 6-DOF en tiempo real mediante un algoritmo híbrido que combina características visuales dispersas, nubes de puntos densas y la extracción del plano de suelo para reducir la deriva. Con esa estimación de pose construye un mapa incremental del entorno y planifica trayectorias seguras hacia un destino, transmitiendo la dirección al usuario a través del chaleco. El sistema fue validado con sujetos ciegos reales en entornos interiores no conocidos previamente y corre a aproximadamente 30 Hz sobre una laptop. La contribución central es demostrar que la combinación de RGB-D, IMU y grilla de ocupación resulta viable para asistencia portátil a ciegos en tiempo real; su limitación más evidente es la dependencia de hardware de cómputo no embebido (laptop).

Zhang et al. [7] avanzan sobre esta línea incorporando una IMU externa de mayor precisión (VN-100) y un algoritmo de odometría visual-inercial potenciado por profundidad denominado DVIO (*Depth-enhanced Visual-Inertial Odometry*). DVIO integra la extracción del plano de suelo con características visuales y mediciones inerciales en un marco de optimización de grafos para estimar la pose 6-DOF a 18 Hz sobre hardware embebido compacto. El sistema opera sobre una plataforma robótica de acompañamiento (*Robotic Navigation Aid*) y tiene como objetivo localizar la plataforma en un plano arquitectónico del edificio para guiar al usuario hacia un destino. Este trabajo muestra que la fusión profunda de datos de profundidad e inercia en un marco de optimización de grafos permite reducir la deriva de odometría en condiciones de iluminación variable y texturas pobres, comparado con los enfoques que tratan la IMU como entrada secundaria.

2.1.4. Producto comercial consolidado: *SuperBrain 1*

Un ejemplo comercial de este tipo de sistemas es el SuperBrain 1, desarrollado por la empresa estonia Seventh Sense OÜ [8]. El dispositivo cuenta con marcado CE (producto comercial certificado) y fue probado con personas con discapacidad visual en distintos países de Europa. Esto muestra que es posible integrar percepción de profundidad, procesamiento embebido y retroalimentación háptica vestible en un dispositivo de uso real.

El dispositivo es un casco portado en la cabeza que integra cámaras estéreo de campo amplio (185° horizontal, 120° vertical), sensores de tiempo de vuelo (*Time-of-Flight*, ToF) para extender el campo de detección lateral, y una cámara térmica que permite la percepción de personas y fuentes de calor. La interfaz de salida consiste en una matriz de puntos de presión sobre la frente del usuario (*haptic mat*), que transmite posición, movimiento y distancia de objetos de forma continua. A diferencia de los arreglos vibrotáctiles convencionales revisados en la

Section 2.1.2, este mecanismo de presión genera una experiencia táctil más rica y precisa. Una función adicional, asistida por inteligencia artificial, implementa un *zoom* inteligente que permite enfocar selectivamente objetos de interés en la escena y representarlos con mayor resolución háptica, capacidad que supera lo previsto en el prototipo desarrollado en esta tesis.

Según la información pública del fabricante, el dispositivo no requiere el uso de las manos, no interfiere con la audición y se porta sobre el cuerpo. Estos atributos permiten analizarlo respecto de los criterios de adopción, aunque la simplicidad de uso requiere evidencia con usuarios y no se infiere únicamente de su descripción comercial. Su costo se encuentra en el orden de miles de euros y su desarrollo demandó más de diez años, esto marca una diferencia importante de alcance respecto del prototipo académico desarrollado en esta tesis. En consecuencia, no se plantea una equivalencia funcional con esta tesis. El *SuperBrain 1* se utiliza como referencia de las capacidades que puede alcanzar una solución madura, mientras que este trabajo estudia un subconjunto acotado: percepción local, salida lógica compacta y ejecución en hardware embebido. Las funciones avanzadas del producto, como el *zoom* con inteligencia artificial y la cámara térmica, se proponen como líneas de trabajo futuro en el Capítulo 11.

2.1.5. Síntesis comparativa de los sistemas revisados

Los sistemas sin RGB-D revisados en la Section 2.1.2 comparten dos limitaciones fundamentales: en primer lugar, utilizan sensores puntuales o de muy bajo número de haces, lo que limita la representación de la distribución tridimensional de obstáculos en el campo visual del usuario. En segundo lugar, esas mediciones aisladas aportan poca información geométrica para estimar el movimiento del usuario por sí solas. Esto no impide obtener odometría mediante una IMU, una cámara u otro sensor adicional, pero los sistemas puntuales aquí revisados no incorporan esas fuentes ni una memoria espacial. Como resultado, el conocimiento sobre el entorno se descarta entre fotogramas y no puede utilizarse para inferir la presencia de obstáculos que hayan salido momentáneamente del campo sensado. Esta doble restricción dificulta la detección de obstáculos a altura de cabeza, la anticipación a geometrías complejas y la posibilidad de implementar lógicas de evitación de colisiones que operen sobre información acumulada.

En contraste, los sistemas con RGB-D incorporan cámaras de profundidad para aumentar la resolución espacial y, en algunos casos, añadir memoria temporal mediante mapeo local. Esta evolución tecnológica se expresa en la Tabla 2.1, que consolida la evaluación de todos los sistemas revisados en este capítulo, desde los basados en sensores puntuales hasta los de cámara de profundidad y el producto comercial *SuperBrain 1*, respecto de los criterios de Dakopoulos y Bourbakis [13] y las dos dimensiones técnicas adicionales. La tabla evidencia la progresión desde soluciones de baja resolución y sin memoria temporal hacia sistemas con percepción densa y acumulación de historial, con el nivel de inversión que cada etapa conlleva.

La clasificación es cualitativa y se limita a las capacidades descritas por cada fuente. Se considera resolución muy baja cuando el sistema entrega una única distancia o dirección, y alta cuando conserva una medición densa del campo observado. La memoria temporal se marca únicamente cuando el trabajo describe una representación persistente del entorno. La última fila corresponde al proyecto global, que integra el módulo de percepción y mapeo desarrollado en esta tesis con la interfaz háptica física implementada en la segunda etapa. Por eso los criterios de adopción se evalúan sobre el dispositivo completo, mientras que la resolución espacial y la memoria temporal provienen del aporte específico de esta tesis.

Tabla 2.1: Comparación de sistemas de asistencia sensorial para personas ciegas según los criterios de Dakopoulos y Bourbakis [13] y dos criterios técnicos adicionales. ✓: satisfecho; ~: parcialmente satisfecho; ×: no satisfecho. La fila «Proyecto global» considera la integración de las dos etapas del proyecto.

Sistema	Manos libres	Oídos libres	Vestible	Simple	Resolución espacial	Memoria temporal
Novich y Eagleman [1]	✓	✓	✓	~	baja	×
Santos et al. [2]	✓	✓	~	✓	muy baja	×
Petsiuk y Pearce [3]	~	✓	✓	✓	muy baja	×
Yang et al. [14]	✓	✓	✓	~	alta	×
Lee y Medioni [6]	✓	✓	✓	~	alta	✓
SuperBrain 1 [8]	✓	✓	✓	✓	alta	~
Proyecto global	✓	✓	✓	✓	media	✓

2.2. Mapeo local y evitación de obstáculos en robótica móvil

La revisión de los dispositivos de asistencia presentada en las secciones anteriores muestra que las limitaciones de resolución y de memoria temporal provienen, en parte, de no explotar las técnicas de mapeo local consolidadas en el campo de la robótica móvil. Esta sección revisa dichos enfoques y sus representaciones de ocupación probabilística, que constituyen el fundamento del módulo de mapeo desarrollado en esta tesis.

2.2.1. Mapas locales reactivos

En robótica móvil, los mapas locales se emplean en estrategias de navegación reactiva para acotar el área actualizada y evitar la dependencia inmediata de un mapa global. Brock y Khatib [15] argumentan que un mapa local centrado en el robot y actualizado a alta frecuencia permite reaccionar ante obstáculos dinámicos sin requerir la coherencia global de un mapa completo del entorno.

2.2.2. Representaciones de ocupación probabilística

La representación canónica para este tipo de mapas es la grilla de ocupación probabilística, introducida por Elfes [11] y formalizada en el marco bayesiano por Thrun, Burgard y Fox [12]. La formulación en escala de log-odds permite actualizar la probabilidad de ocupación de cada celda con costo computacional constante y acumular historial de observaciones, lo cual actúa como filtro de baja frecuencia sobre el ruido del sensor. El trazado de rayos mediante el algoritmo de Bresenham [16] complementa la actualización al marcar explícitamente como libres las celdas atravesadas por el haz antes del punto de impacto.

Existen representaciones volumétricas más expresivas, como OctoMap [17], que extienden la grilla de ocupación 2D al espacio tridimensional mediante octrees y permiten representar obstáculos a distintas alturas con resolución adaptativa. Sin embargo, su mayor fidelidad conlleva un costo computacional significativamente más alto que el de una grilla 2D plana. En plataformas de robótica móvil con cómputo embebido, el *costmap* local bidimensional de Nav2 [18] constituye una alternativa habitual: una grilla 2D de ocupación actualizada a partir de

sensores de profundidad, con capas configurables para inflación de obstáculos y filtrado temporal.

2.2.3. Representación 2D y compensación gravitacional

El mapa local desarrollado en esta tesis adopta los principios de la grilla de ocupación probabilística en escala de log-odds y utiliza una proyección 2D de la nube de puntos sobre el plano horizontal. La discriminación de obstáculos relevantes se resuelve antes de la proyección, mediante alineación gravitacional y filtrado por bandas de altura, conservando en la grilla únicamente la ocupación plana necesaria para la navegación local. De este modo se mantiene una representación compatible con las restricciones de cómputo embebido y con la interfaz háptica de baja resolución utilizada por el sistema.

La alineación gravitacional descrita se basa en la estimación instantánea del vector de gravedad a partir del acelerómetro [19], y es conceptualmente distinta de la odometría visual-inercial [20]-[22]: no integra velocidades ni construye una trayectoria, sino que estima roll y pitch instantáneos a partir de la componente estática de la aceleración.

2.2.4. Estimación del movimiento egocéntrico

El desplazamiento egocéntrico del mapa local ante traslaciones del usuario requiere una estimación del movimiento relativo de la cámara. El giroscopo permite estimar cambios de orientación, pero su integración aislada no observa la traslación y, por lo tanto, no constituye una odometría posicional completa. Para estimar trayectoria pueden emplearse el registro geométrico de nubes o la odometría RGB-D basada en características visuales y profundidad. Cada alternativa presenta compromisos diferentes entre costo computacional, sensibilidad a la geometría o textura de la escena y deriva [19], [23].

Los trabajos de asistencia revisados adoptan soluciones específicas: Lee y Medioni [6] emplean una estimación visual-inercial sobre una computadora portátil, mientras que Zhang et al. [7] usan DVIO con optimización de grafos sobre una plataforma robótica. En esta tesis se evaluó inicialmente el registro ICP de nubes y se adoptó finalmente `rgbd_odometry`, del paquete `rtabmap_odom` de RTAB-Map. El filtro Madgwick entrega una orientación obtenida de acelerómetro y giroscopo como entrada auxiliar; no se presenta esta configuración como una VIO estrechamente acoplada, porque RTAB-Map no optimiza conjuntamente los estados visuales e inerciales en el esquema utilizado. La Tabla 2.2 resume las alternativas relevantes y el papel de cada fuente.

2.3. Limitaciones identificadas y aporte de la tesis

A partir de los trabajos revisados se identifican cuatro limitaciones relevantes:

1. **Resolución espacial insuficiente:** los ETA dominantes emplean sensores puntuales que no permiten representar la distribución tridimensional de obstáculos en el campo visual del usuario [2], [3], [13]. En particular, la ausencia de cobertura vertical impide detectar obstáculos a la altura de la cabeza, como ramas, cables o señales colgantes, que escapen al alcance del bastón blanco y requieren información espacial en el eje vertical.
2. **Ausencia de acumulación temporal de observaciones:** los sistemas basados en cámara de profundidad como el de Yang et al. [14] generan retroalimentación directamente desde el fotograma instantáneo, sin conservar historial de observaciones. La consecuencia más relevante no es la incapacidad de resolver oclusiones en profundidad, ya que una

Tabla 2.2: Alternativas consideradas para estimar el movimiento del mapa local y configuración adoptada con `rtabmap_odom`.

Modo	Fuente	Implementación	Compromisos
Orientación inercial	Acelerómetro y giróscopo	Madgwick [24]	Aporta orientación; no estima por sí sola la traslación
Registro geométrico	Nube de profundidad	<code>icp_odometry</code> de RTAB-Map [23]	Depende de geometría suficiente y de una inicialización válida; alternativa descartada
Odometría RGB-D adoptada	Imagen RGB, profundidad y orientación IMU	<code>rgbd_odometry</code> de <code>rtabmap_odom</code> [23]	Usa características y profundidad métrica; puede degradarse ante texturas pobres o movimientos rápidos

representación 2D con memoria temporal como la de esta tesis tampoco reconstruye zonas no visibles, sino la pérdida de información sobre obstáculos que han salido del campo de visión de la cámara. Un usuario que gira la cabeza o que avanza dejando un objeto fuera del encuadre pierde instantáneamente ese dato; sin acumulación temporal no es posible implementar lógicas de evitación que operen sobre el historial reciente del entorno, lo que limita estructuralmente la utilidad del sistema para anticipar colisiones con objetos que ya no están siendo observados en ese instante.

3. **Costo y accesibilidad del hardware:** el producto *SuperBrain 1* [8] muestra que la percepción densa y la realimentación háptica vestibular pueden integrarse en una solución comercial, pero su costo se encuentra en el orden de miles de euros y su maduración demandó más de diez años. Entre los trabajos académicos revisados, Lee y Medioni [6] emplean una computadora portátil y Zhang et al. [7] una plataforma robótica embebida. Estos antecedentes no permiten trasladar directamente sus resultados a un dispositivo corporal basado en Raspberry Pi 5; por ello, esta tesis debe caracterizar experimentalmente el costo computacional de su propia configuración.
4. **Necesidad de baja latencia:** las métricas temporales publicadas por los trabajos revisados no son directamente comparables, porque pueden referirse a frecuencia del sensor, tiempo de procesamiento o tasa de actualización, y no necesariamente al recorrido completo desde la adquisición hasta la advertencia. En un sistema de asistencia utilizado durante la marcha, la información debe actualizarse con suficiente rapidez para representar el obstáculo antes de que el usuario alcance su posición. Por ello, el retardo debe analizarse en relación con la velocidad de desplazamiento, la distancia de detección y el tiempo disponible para reaccionar, no solo con la frecuencia nominal de un nodo. Esta tesis incluye la latencia extremo a extremo entre las métricas del Capítulo 7; el umbral de aceptación se definirá antes de interpretar los resultados.

La presente tesis aborda estas cuatro brechas mediante la adaptación y validación de técnicas de percepción y mapeo local propias de la robótica móvil al dominio de la asistencia sensorial portátil. A diferencia de Lee y Medioni [6], cuyo sistema apunta a la navegación global con planificación de rutas, el enfoque adoptado en esta tesis se centra en la conciencia local de obstáculos y en la generación de una representación háptica compacta, priorizando la ejecución sobre una plataforma de cómputo embebido. Se adopta una representación de baja resolución de 5 filas $\times$ 10 columnas, compatible con arreglos de actuadores hápticos de baja complejidad y con las restricciones de ancho de banda y cómputo de plataformas embebidas, con un compromiso explícito entre resolución espacial y frecuencia de actualización en línea con la discusión de la Section 2.2.1. La cobertura vertical se obtiene durante el procesamiento 3D, antes de proyectar los obstáculos; la grilla 2D conserva ocupación sobre el plano horizontal y aporta memoria temporal, pero no mantiene información de altura. La plataforma Raspberry Pi 5 permite evaluar el costo computacional sobre hardware embebido, y la configuración de movimiento adoptada se describe en la Section 2.2.4.

Capítulo 3

Marco teórico

Los fundamentos se agrupan según la versión en la que cada herramienta se incorpora al sistema; la motivación y el alcance de v0, v1 y v2 se establecen en la Sección 1.3.3.

Este capítulo explica el significado de las representaciones y transformaciones utilizadas. La correspondencia concreta entre nodos, mensajes de entrada y mensajes de salida se muestra, para v0, v1 y v2, en las Figuras 4.4, 4.5 y 4.6 del capítulo de arquitectura. Esta separación permite presentar los fundamentos sin repetir la topología de software y mantener la relación entre cada concepto y los datos que circulan por ROS 2.

El procesamiento realizado por el sistema consiste en una secuencia de transformaciones y reducciones de información. Una imagen de profundidad asigna una distancia a cada píxel. Al combinar esa distancia con la calibración de la cámara, cada medición puede convertirse en un punto tridimensional. Un conjunto de esos puntos forma una *nube de puntos*; esta representación no implica continuidad ni una superficie reconstruida, sino una colección de posiciones 3D expresadas respecto de un sistema de coordenadas. Después de descartar las mediciones que no representan obstáculos, los puntos se reducen a una grilla frontal o se acumulan en un mapa local. Finalmente, la distancia y el riesgo se traducen a intensidades lógicas. La Figura 3.1 reúne estas representaciones y muestra la información que se conserva o se descarta en cada transformación.

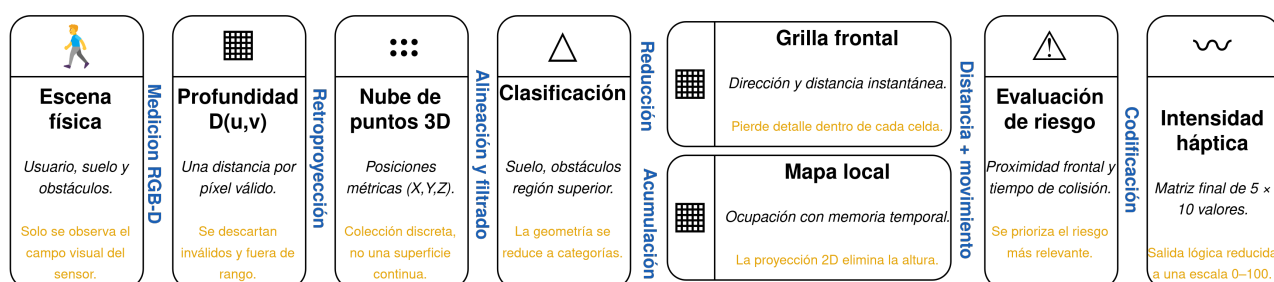


Figura 3.1: Transformaciones y reducciones de información desde la escena física hasta la intensidad háptica. Las anotaciones naranjas indican la información descartada o resumida en cada etapa.

3.1. Fundamentos de la versión v0: percepción directa, ROS 2 y salida háptica

La versión v0 establece la cadena mínima del sistema: adquirir profundidad con la Intel RealSense D435i, reducir esa información a una representación compacta, convertirla a comandos de intensidad háptica y preparar su transmisión hacia el futuro hardware de actuación. Esta sección presenta los fundamentos teóricos que sostienen esa primera etapa, desde la sustitución sensorial hasta la comunicación serie embebida. El objetivo es mostrar cómo una medición densa del entorno puede convertirse en una señal discreta y de baja latencia sin perder la estructura espacial necesaria para la asistencia a la movilidad.

3.1.1. Sustitución sensorial y neuroplasticidad

El principio de sustitución sensorial fue establecido experimentalmente por Bach-y-Rita et al. [5]: una cámara proyectaba patrones de estimulación sobre la espalda del usuario mediante una matriz de 20×20 actuadores mecánicos, permitiendo a personas ciegas reconocer objetos y figuras a través del tacto. Este resultado demostró que el cerebro puede reorganizarse para interpretar señales de modalidades no originales cuando recibe información espacial consistente, fenómeno conocido como neuroplasticidad sensorial.

Para esta tesis, la relevancia de este antecedente es doble. En primer lugar, justifica que una escena visual pueda codificarse como estímulo háptico si se preserva una correspondencia espacial estable. En segundo lugar, orienta el diseño hacia representaciones compactas y repetibles: la interfaz no necesita reconstruir toda la imagen, sino entregar patrones táctiles consistentes que el usuario pueda aprender a interpretar.

3.1.2. Percepción RGB-D y geometría de cámara

Este bloque vincula el principio de medición del sensor con la representación geométrica que utiliza el pipeline. Primero se explica cómo la D435i obtiene profundidad y bajo qué condiciones esa medición puede degradarse; luego se definen los marcos, la pose y el modelo de proyección que permiten transformar un píxel de profundidad en un punto tridimensional. Esta secuencia separa dos fuentes de error diferentes: las limitaciones físicas de la medición y una interpretación incorrecta de sus coordenadas.

Principio físico de la cámara Intel RealSense D435i

Los sensores RGB-D combinan información visual convencional con una medición de profundidad. En la familia Intel RealSense D400, y en particular en la D435i, la profundidad se obtiene mediante estereoscopía infrarroja activa: dos cámaras IR observan la escena y un proyector de textura facilita la correspondencia entre vistas en regiones con bajo contraste [4], [25]. El resultado es una imagen de profundidad $D(u, v)$ donde cada píxel codifica la distancia al plano de la cámara; los valores centinela $D = 0$ y $D = 65535$ indican ausencia de medición válida.

Bajo imágenes rectificadas, la correspondencia entre un píxel de la vista izquierda u_L y el píxel correspondiente de la vista derecha u_R define la disparidad $d = u_L - u_R$. Aplicando semejanza de triángulos en el sistema estereoscópico se obtiene:

$$Z = \frac{f B}{d}, \quad (3.1)$$

Aquí, Z es la profundidad del punto respecto del sistema estéreo, B es la separación entre los centros ópticos de las cámaras y f es la distancia focal expresada en píxeles [26], [27]. La ecuación 3.1 muestra que el error de profundidad crece al disminuir la disparidad, por lo que las mediciones lejanas son más sensibles al ruido y a errores de correspondencia.

La Figura 3.2 representa cada imagen rectificadas únicamente sobre su eje horizontal. Las variables u_L y u_R son las coordenadas, en píxeles, donde se observa el mismo punto en las imágenes izquierda y derecha; su diferencia es la disparidad d . Los planos de imagen se dibujan delante de las cámaras porque se utiliza el modelo *pinhole* con un plano virtual. En la cámara física, el sensor se encuentra detrás del centro óptico y forma una imagen invertida; trasladar conceptualmente ese plano hacia delante permite representar una imagen derecha sin modificar la geometría de proyección. Por ello, el plano mostrado no representa un segundo sensor ni su posición física. La longitud rotulada f representa la distancia focal del modelo y, para combinarla con una disparidad medida en píxeles en la Ecuación 3.1, se utiliza su valor calibrado también en píxeles.

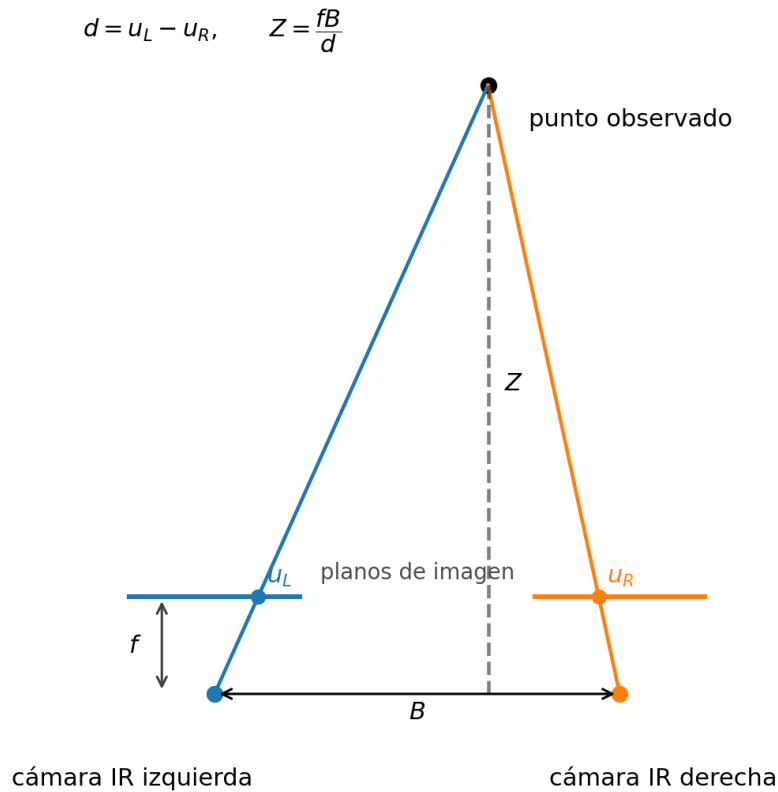


Figura 3.2: Geometría estéreo rectificadas. Un mismo punto se proyecta en las coordenadas horizontales u_L y u_R de los planos virtuales de imagen; su diferencia define la disparidad d .

La D435i publica, además de la imagen de profundidad, flujos de color, profundidad alineada a color, parámetros intrínsecos de cámara y señales inerciales. La distancia focal ya interviene en el cálculo estéreo que la cámara realiza internamente para producir $D(u, v)$. Sin embargo, obtener la profundidad Z de un píxel no equivale a conocer la posición tridimensional completa del punto: para recuperar también sus coordenadas X e Y es necesario relacionar la posición (u, v) dentro de la imagen con la geometría calibrada de la cámara. Esta relación, y la razón por la que v_0 y v_1 utilizan entradas diferentes, se formalizan luego mediante el modelo *pinhole*.

Antes de expresar esa relación es necesario precisar respecto de qué origen y ejes se describen las coordenadas obtenidas.

Sistemas de coordenadas, pose y transformaciones

Una misma posición física adquiere coordenadas distintas según el origen y los ejes elegidos. Ese origen con sus tres ejes constituye un *marco de referencia*. La pose de un marco B respecto de otro A combina una rotación R_{AB} y una traslación $\mathbf{t}_{AB}$. Para un punto $\mathbf{p}_B$ expresado en B , sus coordenadas en A son

$$\mathbf{p}_A = R_{AB}\mathbf{p}_B + \mathbf{t}_{AB}. \quad (3.2)$$

La rotación cambia la orientación de los ejes y la traslación cambia su origen. En v0, las mediciones permanecen en el marco óptico de la cámara. Las versiones posteriores encadenan transformaciones para pasar primero a un marco alineado con la gravedad y luego a un marco común en el que se acumulan observaciones tomadas en instantes distintos. ROS 2 representa estas relaciones mediante el árbol TF; las convenciones concretas empleadas por v2 se presentan en la Sección 3.3.1.

Modelo de cámara *pinhole* y proyección inversa

El modelo *pinhole* relaciona las coordenadas de imagen con las coordenadas espaciales. Un punto $\mathbf{P} = (X, Y, Z)^T$ expresado en el marco de la cámara se proyecta al píxel (u, v) mediante:

$$\begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \frac{1}{Z} K \begin{pmatrix} X \\ Y \\ Z \end{pmatrix}, \quad K = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}. \quad (3.3)$$

En la ecuación 3.3, f_x y f_y son las distancias focales en píxeles, mientras que (c_x, c_y) es el centro de proyección. La operación inversa, necesaria para reconstruir una nube 3D a partir de una imagen de profundidad, queda:

$$X = \frac{(u - c_x) Z}{f_x}, \quad Y = \frac{(v - c_y) Z}{f_y}, \quad Z = D(u, v) \times 10^{-3} \text{ m}. \quad (3.4)$$

Los parámetros de K se obtienen del mensaje `sensor_msgs/CameraInfo`, lo que evita fijar intrínsecos a mano y mantiene coherencia con el perfil activo de resolución y calibración [26].

La diferencia entre ambas versiones no consiste, entonces, en que v0 prescinda de la calibración de la cámara, sino en dónde se utiliza. La RealSense la aplica internamente para entregar $D(u, v)$ a ambas versiones; solo v1 vuelve a consultar los intrínsecos porque necesita obtener X e Y . En v0, cada píxel permanece en coordenadas de imagen y se agrega directamente dentro de su ROI. En v1 puede procesarse un píxel cada s posiciones en ambas direcciones. Este *pixel stride* reduce aproximadamente la cantidad de proyecciones por un factor s^2 ; no debe confundirse con el *voxel grid*, que agrupa puntos después de proyectarlos al espacio tridimensional.

Limitaciones por superficies especulares e iluminación intensa

La estimación estéreo supone que los píxeles comparados en ambas cámaras observan el mismo punto físico mediante trayectorias ópticas directas y que su apariencia local es suficientemente similar. Esta hipótesis se cumple razonablemente bien en superficies difusas, pero se debilita en superficies especulares o semiespeculares, como pisos pulidos, baldosas brillantes, metal, vidrio o zonas húmedas. En esos casos la energía infrarroja emitida por el proyector de textura, o la

luz proveniente de una fuente externa, se refleja de manera direccional y puede llegar a cada cámara desde caminos ópticos distintos.

El problema no es solamente la ausencia de datos. Si el algoritmo de correspondencia no encuentra textura confiable o detecta saturación, el resultado esperable es una medición inválida. Sin embargo, también puede ocurrir una correspondencia falsa pero internamente consistente. Un reflejo sobre el piso puede aparecer en ambas cámaras IR con posiciones compatibles con cierta disparidad d ; al aplicar la ecuación 3.1, el sistema interpreta esa disparidad como una profundidad real, aunque el patrón observado corresponda a una reflexión, a una imagen virtual o a luz parásita.

Al retroproyectar mediante la ecuación 3.4, esa medición se convierte en un punto 3D que no pertenece a un obstáculo físico. Para una interfaz háptica, este fenómeno tiende a manifestarse como falso positivo si la agregación por celda conserva una lectura cercana aislada. La mitigación requiere tratar estas lecturas como una limitación física del sensor y no como un error puramente algorítmico: descartar inválidos, exigir soporte espacial mínimo, incorporar persistencia temporal y verificar coherencia con el plano de suelo [4], [25].

3.1.3. ROS 2 como infraestructura de comunicación distribuida

ROS 2 proporciona mecanismos para dividir un sistema robótico en componentes y comunicar esos componentes dentro de una computadora o a través de una red [10]. Esta separación permite que adquisición, percepción, mapeo, visualización y salida se desarrollen como módulos distintos. La arquitectura concreta del sistema se presenta en el Capítulo 4; aquí se introducen únicamente los conceptos necesarios para comprender cómo circulan los datos y qué factores pueden afectar su entrega y latencia.

Modelo de comunicación e interfaces

Un *nodo* es una unidad de procesamiento con una responsabilidad definida. Los nodos forman un grafo y se comunican mediante interfaces tipadas. Los *tópicos* implementan comunicación asíncrona de tipo publicador-suscriptor: uno o más nodos publican mensajes y los suscriptores los reciben sin que ambas partes deban conocerse entre sí. Los *servicios* se usan para una solicitud con respuesta breve, mientras que las *acciones* representan tareas prolongadas que pueden informar avance o cancelarse [28]. El pipeline emplea principalmente tópicos porque imágenes, mediciones inerciales, nubes, odometría y mapas son flujos continuos; las órdenes puntuales, como iniciar o detener una grabación, se ajustan mejor a un servicio.

El tipo del mensaje define la estructura y las unidades de la información intercambiada. Los tipos estándar permiten integrar drivers y herramientas existentes, mientras que los mensajes propios expresan contratos específicos del sistema. En ambos casos, una interfaz explícita evita que el significado de cada campo dependa de acuerdos implícitos entre productor y consumidor.

Capas RCL, RMW y DDS

El código de un nodo no accede directamente a DDS. Los nodos escritos en C++ o Python utilizan las bibliotecas cliente `rclcpp` o `rclpy`; estas se apoyan en la biblioteca común `rcl`, que delega la comunicación en la interfaz `RMW` (*ROS Middleware*). Una implementación de `RMW` adapta esa interfaz al middleware elegido. En este sistema, `rmw_cyclonedds_cpp` realiza esa adaptación para Cyclone DDS [29].

DDS se ocupa del descubrimiento de participantes, la serialización y el transporte de los datos. El descubrimiento distribuido permite que los nodos se encuentren sin un servidor maestro

central. El identificador de dominio, `ROS_DOMAIN_ID`, limita qué participantes pueden descubrirse entre sí. De esta forma, el procesamiento puede ejecutarse en la Raspberry Pi y la visualización en otra computadora, siempre que exista conectividad y ambas pertenezcan al mismo dominio. La Figura 3.3 resume las capas de software que intervienen desde la aplicación hasta el sistema operativo.

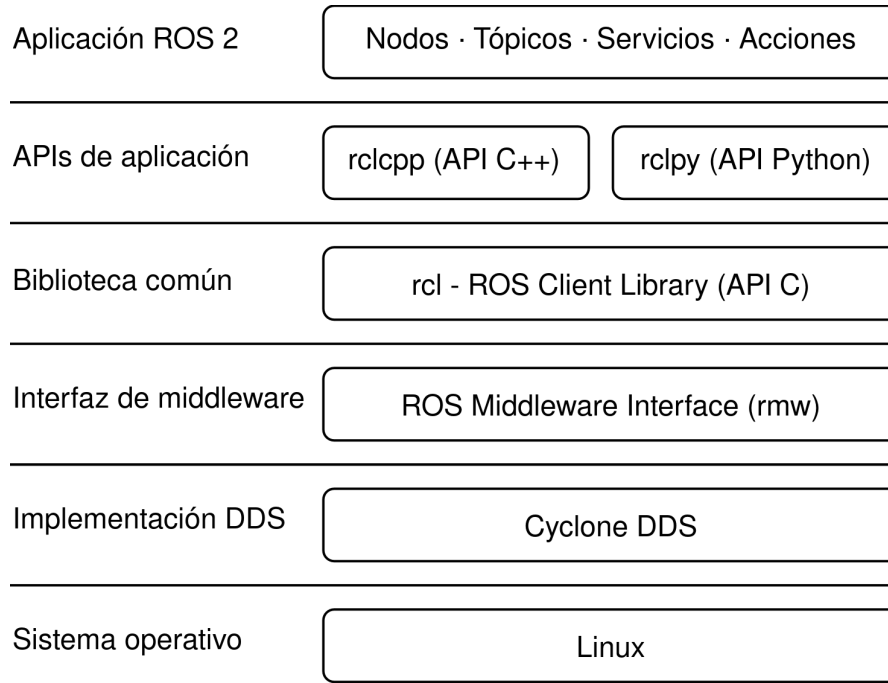


Figura 3.3: Capas de comunicación de ROS 2 empleadas en el sistema. Los nodos acceden a las funciones de ROS 2 mediante `rclcpp` o `rclpy`, construidas sobre la biblioteca común `rcl`. La interfaz `RMW` desacopla estas bibliotecas del middleware DDS; en este trabajo se utiliza Cyclone DDS sobre Linux.

Calidad de servicio y comportamiento temporal

Cada publicador y suscriptor posee un perfil de calidad de servicio o QoS. Este perfil no mejora el contenido del mensaje: establece cómo debe entregarse. La política de confiabilidad puede ser *best effort*, que admite pérdidas para evitar retransmisiones, o *reliable*, que intenta garantizar la entrega. La política de historial y la profundidad determinan cuántas muestras se conservan. La durabilidad *volatile* entrega solo publicaciones nuevas; con *transient local*, el publicador conserva muestras para suscriptores que se conectan después [30]. Los perfiles ofrecido y solicitado deben ser compatibles para que se establezca la comunicación.

La elección depende de la función del dato. En un flujo rápido de sensores suele ser preferible procesar la muestra más reciente antes que esperar la retransmisión de una muestra antigua. En cambio, la odometría requiere una entrega más confiable, y estados de baja frecuencia como un mapa o una configuración pueden necesitar que un suscriptor nuevo reciba el último valor disponible. Por eso el sistema combina `SensorDataQoS`, perfiles confiables y durabilidad *transient local* según el tópico.

Cuando llega un mensaje, su callback queda disponible para ser ejecutado. Si el nodo todavía está procesando otra tarea, la muestra permanece en la cola hasta que el ejecutor pueda atenderla. Una cola corta puede descartar datos durante una sobrecarga; una cola larga reduce pérdidas, pero puede hacer que se procesen datos antiguos. Por lo tanto, la latencia extremo a extremo

incluye no solo transporte y cálculo, sino también espera en colas. ROS 2 ofrece mecanismos para configurar este comportamiento, pero no garantiza por sí solo ejecución en tiempo real.

Registro y reproducción de datos con rosbag2

`rosbag2` registra mensajes ROS 2 junto con su tópico, tipo y marca temporal, y permite publicarlos nuevamente conservando la secuencia temporal de la captura [31]. Así, una misma escena puede procesarse con distintas configuraciones sin repetir la adquisición física. En una comparación experimental esto mantiene constantes las mediciones de entrada y reduce variaciones debidas al movimiento del usuario, la posición de los obstáculos o la iluminación.

La reproducción no vuelve determinista toda la ejecución. La planificación del sistema operativo, la espera en colas y el tiempo de los callbacks todavía pueden variar entre corridas. Además, los perfiles QoS de grabación y reproducción deben ser compatibles con los nodos evaluados. Por ello, el rosbag controla la entrada del experimento, mientras que las configuraciones, la velocidad de reproducción y la cantidad de repeticiones deben fijarse por separado.

3.1.4. Representación compacta y salida embebida

La salida de la cámara contiene más información que la necesaria para accionar una interfaz háptica de baja resolución. La cadena reduce esa información en etapas: resume la imagen densa, conserva la distribución angular de los obstáculos, transforma distancia en intensidad y serializa el resultado. Cada etapa cambia la representación, pero mantiene explícita la correspondencia espacial entre una región del entorno y una celda de salida.

Reducción espacial de la imagen de profundidad

Una imagen de profundidad a resolución nativa contiene cientos de miles de píxeles por fotograma, pero esa densidad no puede trasladarse de forma directa a una interfaz háptica corporal. La limitación principal no es solamente computacional o de transmisión, sino perceptual: el usuario no puede distinguir de manera confiable estímulos hápticos arbitrariamente cercanos ni interpretar una matriz demasiado densa como si fuera una imagen visual. Por este motivo, la escena debe reducirse a una representación espacial de baja dimensión que conserve la ubicación aproximada y la urgencia de los obstáculos. En v0, esta reducción se realiza particionando la imagen en regiones de interés y calculando estadísticas por región.

La imagen de dimensiones $W \times H$ se divide en una grilla de $R \times C$ regiones de interés (ROI). Las fronteras de cada ROI se calculan mediante división entera uniforme:

$$x_c^{(k)} = \left\lfloor \frac{k W}{C} \right\rfloor, \quad k = 0, \dots, C; \quad y_r^{(j)} = \left\lfloor \frac{j H}{R} \right\rfloor, \quad j = 0, \dots, R. \quad (3.5)$$

La ecuación 3.5 distribuye el error de cuantización entre celdas, de modo que regiones adyacentes difieran a lo sumo en un píxel [32].

Para cada ROI(r, c) se recorren los píxeles válidos dentro del rango $[z_{\min}, z_{\max}]$ y se acumulan:

$$\bar{z}_{r,c} = \frac{1}{N_{r,c}} \sum_{(u,v) \in \text{ROI}_{r,c}} z_{u,v}, \quad z_{r,c}^{\min} = \min_{(u,v)} z_{u,v}, \quad N_{r,c} = \#\{\text{píxeles válidos}\}. \quad (3.6)$$

Junto con el máximo $z_{r,c}^{\max}$, estas estadísticas describen distintos aspectos de una región: el mínimo conserva la superficie medida más cercana, la media resume el conjunto de lecturas válidas y el máximo conserva la más lejana [17].

Por ejemplo, para las profundidades $[1,20; 0; 1,35; 1,10]$ m de una celda, el cero se descarta como medición inválida y se obtiene $N = 3$, $z^{\min} = 1,10$ m, $z^{\max} = 1,35$ m y $\bar{z} \simeq 1,22$ m. Este ejemplo muestra cómo varios píxeles de la imagen quedan resumidos en las estadísticas de una única celda de **DepthGrid**; la selección del estadístico empleado para generar la intensidad se presenta como decisión de diseño en la Sección ??.

La Figura 3.4 muestra esta reducción sobre un fotograma real. La misma ROI se destaca en la imagen original, en su ampliación y en la posición correspondiente de la grilla. Los píxeles negros representan mediciones inválidas o fuera del rango operativo. Para visualizar la grilla completa se representa el campo mínimo de cada celda, mientras que el recuadro de la ROI informa también cantidad, media y máximo. La coincidencia indicada en la figura verifica que esos valores fueron recalculados desde la imagen emparejada y coinciden con el mensaje **DepthGrid** publicado.

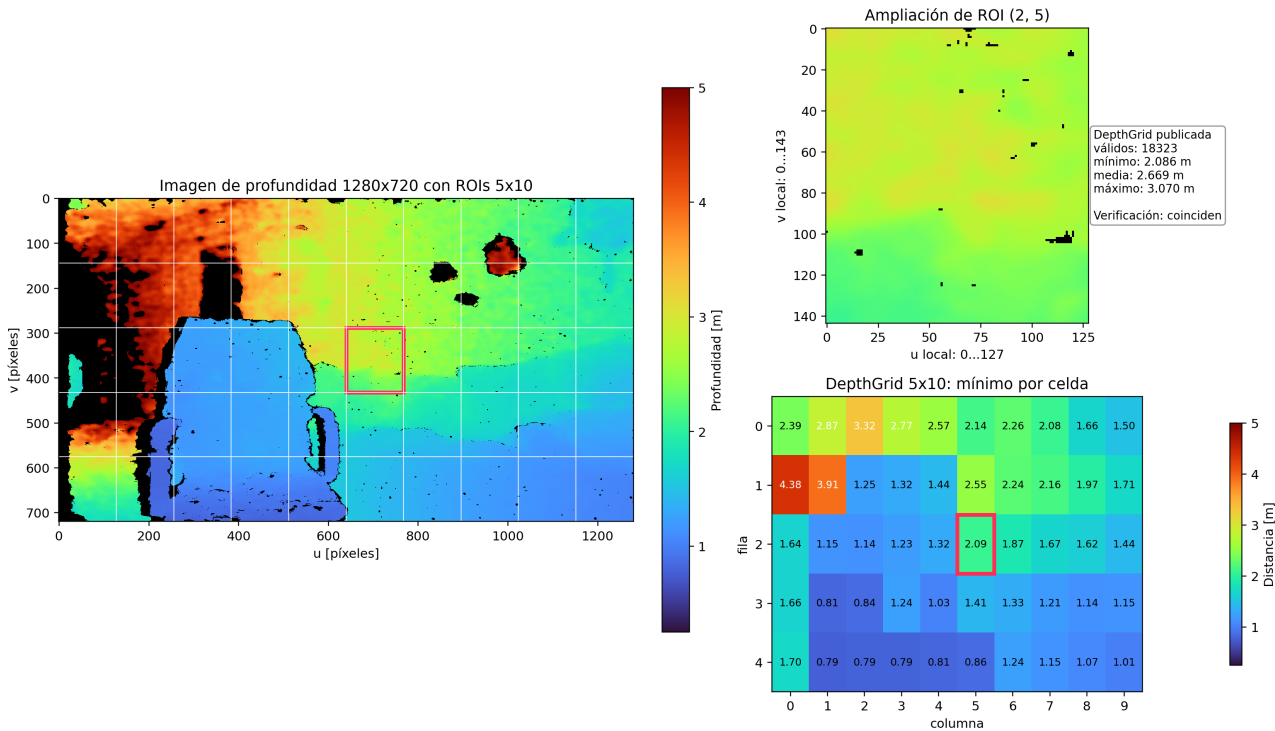


Figura 3.4: Reducción espacial de una imagen de profundidad de 1280×720 píxeles a una **DepthGrid** de 5 filas por 10 columnas. La captura proviene de la ejecución de v0.

Codificación angular de la nube filtrada

Aunque v0 reduce directamente la imagen de profundidad en ROIs de imagen, las versiones v1 y v2 construyen la grilla a partir de una nube de obstáculos ya filtrada. En ese caso, la partición horizontal puede definirse en ángulo, no solo en coordenada de imagen. Si un punto tiene coordenadas (x, z) en el plano horizontal de la cámara, su ángulo lateral es:

$$\beta = \text{atan2}(x, z). \quad (3.7)$$

La apertura horizontal activa se divide en columnas, y cada punto se asigna a la columna correspondiente según β . La distancia usada para codificar proximidad es radial:

$$d = \sqrt{x^2 + z^2}. \quad (3.8)$$

La ecuación 3.8 evita subestimar obstáculos laterales: usar solo z puede hacer que dos obstáculos con distinta separación lateral parezcan equivalentes si comparten la misma componente frontal.

Para mantener fija la interfaz aunque cambie la apertura activa, puede acumularse primero en una grilla angular intermedia más fina y reducir después el intervalo seleccionado a las columnas de salida. Sobre el eje vertical se procede de forma análoga mediante bandas métricas. La convención de orden debe mantenerse estable para que una misma celda corresponda siempre a una misma zona corporal.

La Figura 3.5 sigue un único punto para mostrar su asignación horizontal, su asignación vertical y la reducción final por celda. La aplicación del mismo criterio sobre una nube completa y su correspondencia con el mensaje publicado se presenta en la Figura ?? del capítulo de método.

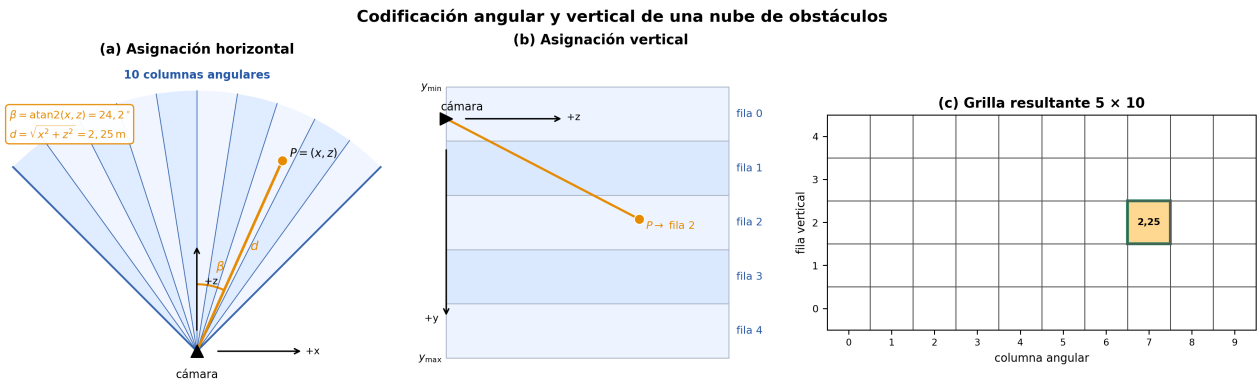


Figura 3.5: Codificación geométrica de una nube de obstáculos. El ángulo lateral determina la columna, la coordenada vertical determina la fila y cada celda conserva la distancia radial mínima de los puntos asignados.

Mapeo de distancia a comando de intensidad

Para traducir la distancia por celda a una señal háptica se emplea una función lineal invertida y saturada. La proximidad del obstáculo debe producir mayor intensidad, con saturación en los extremos del rango útil:

$$I_i = \begin{cases} I_{\text{máx}} & \text{si } z_i^{\text{mín}} \leq z_{\text{mín}}, \\ 0 & \text{si } z_i^{\text{mín}} \geq z_{\text{máx}}, \\ \frac{z_{\text{máx}} - z_i^{\text{mín}}}{z_{\text{máx}} - z_{\text{mín}}} \cdot I_{\text{máx}} & \text{en otro caso.} \end{cases} \quad (3.9)$$

En la ecuación 3.9, $z_{\text{mín}}$ y $z_{\text{máx}}$ delimitan exclusivamente el rango de conversión; $I_{\text{máx}}$ es la intensidad máxima e I_i es la intensidad lógica asociada a la celda i . El rango aceptado por el sensor puede ser mayor que el rango que produce estimulación: una medición puede ser válida y, sin embargo, quedar suficientemente lejos para generar intensidad nula. Los límites y la representación de una celda sin evidencia son decisiones del método especificadas en la Sección ??.

La intensidad lógica no es todavía el ciclo de trabajo aplicado a un motor. La interfaz UART

posterior limita I_i al intervalo $[0, 100]$ y la escala mediante el máximo configurado $D_{\text{máx}}$:

$$D_i = \begin{cases} \left\lfloor \frac{\text{round}(\text{sat}_{[0,100]}(I_i)) D_{\text{máx}}}{100} \right\rfloor, & \text{si la celda está activa,} \\ 0, & \text{si la celda está inactiva.} \end{cases} \quad (3.10)$$

Esta segunda operación no vuelve a interpretar la distancia: solo adapta el comando ya calculado al límite de salida y lo prepara para su serialización.

La Figura 3.6 representa esta diferencia entre el rango aceptado por percepción y el rango de conversión háptica. Los tramos constantes muestran las saturaciones matemáticas de la función.

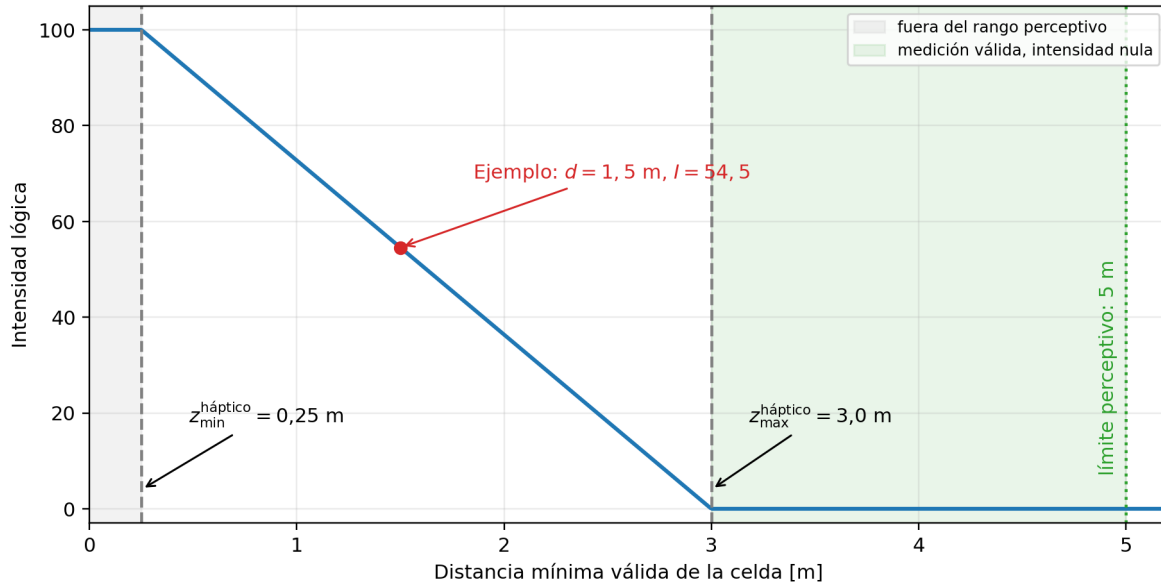


Figura 3.6: Conversión lineal, invertida y saturada de distancia mínima válida a intensidad lógica, distinguiendo el rango de percepción del rango que produce estimulación.

Comunicación serie embebida

La transmisión prevista entre la computadora de placa simple y el futuro controlador de actuación requiere definir un protocolo de enlace. UART (*Universal Asynchronous Receiver-Transmitter*) es una opción habitual en sistemas embebidos por su disponibilidad y simplicidad. En modo 8N1, cada byte requiere 10 bits físicos: un bit de inicio, ocho bits de datos y un bit de parada. Por lo tanto, para una trama de longitud L_{trama} y baudrate B :

$$t_{\text{tx}} = \frac{10 \cdot L_{\text{trama}}}{B}. \quad (3.11)$$

La ecuación 3.11 permite verificar que la transmisión no comprometa el período del lazo háptico.

Existen dos enfoques de codificación. En ASCII, los valores se representan como caracteres decimales separados por espacios y terminados con `\n`; es legible y fácil de depurar, pero menos eficiente. En binario compacto, cada intensidad puede codificarse en un `uint8`; al agregar longitud y CRC-8 se obtiene una trama más robusta y eficiente [33]. La elección depende del ancho de banda, la facilidad de inspección y la necesidad de detectar errores. El formato, baudrate y alcance físico adoptados se describen en las Secciones 4.4.1 y ??; aquí interesa únicamente que el tiempo de serialización forma parte del presupuesto de latencia del sistema.

3.2. Fundamentos de la versión v1: orientación, gravedad y suelo

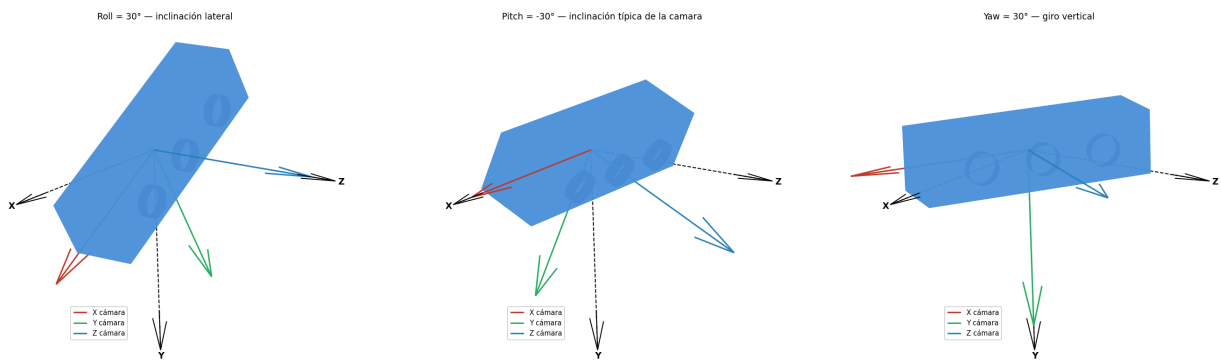
La versión v1 parte de la solución funcional establecida en v0 y conserva su salida compacta, pero mejora la interpretación previa de las mediciones al modificar la fuente de la grilla: en lugar de codificar profundidad cruda, primero proyecta la imagen a una nube 3D, la alinea con la gravedad y separa suelo, obstáculos y una región superior excluida. La teoría de esta etapa combina representaciones de orientación, filtrado inercial y estimación robusta de planos. En esta versión no se utiliza Madgwick de forma deliberada: la decisión de diseño es estimar solamente la vertical necesaria para remover el suelo, evitando incorporar una estimación completa de orientación cuando no aporta a la salida háptica y puede aumentar latencia, dependencias y costo computacional.

3.2.1. Orientación y alineación gravitacional

La clasificación de suelo y obstáculos requiere expresar la nube respecto de la vertical, no respecto de una cámara cuya inclinación cambia con la cabeza del usuario. Para fundamentar esa transformación se presentan primero las convenciones angulares y la limitación de los ángulos de Euler; después se describe la rotación mediante cuaterniones, la estimación de la vertical con el acelerómetro y el filtrado necesario para que las aceleraciones propias de la marcha no se interpreten directamente como cambios de orientación.

Ángulos de cámara y convención óptica

La orientación de la cámara puede describirse mediante ángulos de Euler: *roll* (ϕ), *pitch* (θ) y *yaw* (ψ). En la convención óptica de la D435i, x apunta hacia la derecha de la imagen, y hacia abajo y z hacia adelante. En esa convención, roll corresponde a la rotación alrededor de z , pitch alrededor de x e yaw alrededor de y , como se muestra en la Figura 3.7.



(a) Roll = 30°: rotación alrededor de z . (b) Pitch = -30°: rotación alrededor de x . (c) Yaw = 30°: rotación alrededor de y .

Figura 3.7: Ángulos de Euler en la convención óptica de la cámara. Cada rotación se representa alrededor de un eje fijo del sensor.

La Figura 3.7 también permite distinguir dos funciones diferentes. Roll y pitch modifican la relación entre la nube y el plano de suelo, por lo que deben compensarse antes de segmentar.

En cambio, yaw rota la cámara alrededor de la vertical y no cambia qué puntos pertenecen al suelo; su importancia aparece en v2, cuando se acumulan observaciones en un marco común.

Singularidad de Euler y cuaterniones

Los ángulos de Euler presentan una singularidad conocida como *gimbal lock*: cuando $\theta = \pm 90^\circ$, dos ejes de rotación quedan alineados y se pierde un grado de libertad. La Figura 3.8 muestra una configuración alcanzable cuando la cámara apunta casi verticalmente hacia abajo.

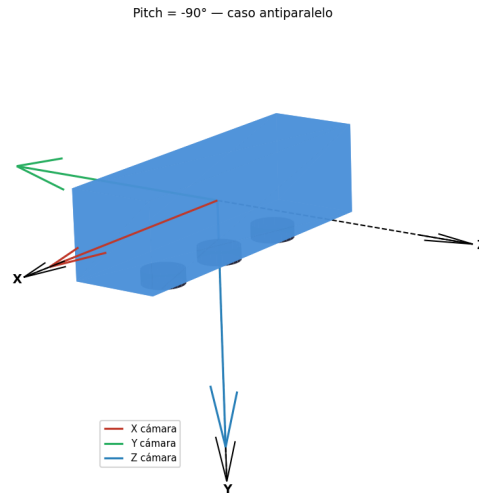


Figura 3.8: Singularidad *gimbal lock*: con pitch = -90° la cámara apunta verticalmente hacia abajo y los ejes de roll y yaw quedan alineados.

Para evitar esa singularidad se usa una representación mediante cuaterniones unitarios. Un cuaternión se define como:

$$q = q_w + q_x \mathbf{i} + q_y \mathbf{j} + q_z \mathbf{k}, \quad (3.12)$$

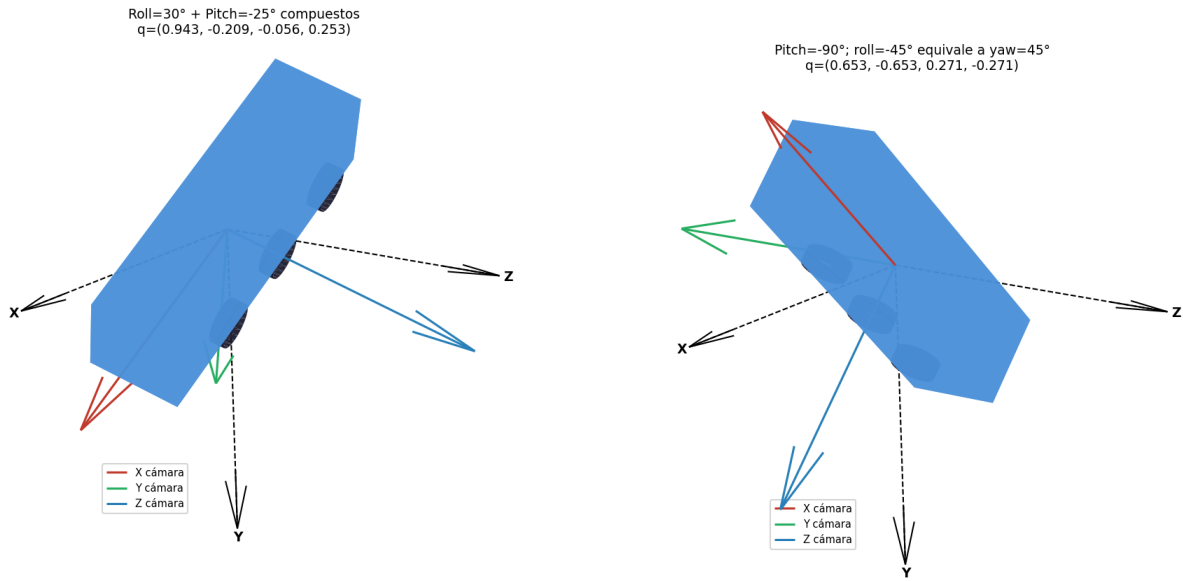
donde q_w es la parte escalar y $(q_x, q_y, q_z)^T$ la parte vectorial. Si $\|q\| = 1$, representa una rotación de ángulo θ alrededor de un eje unitario $\hat{\mathbf{n}}$:

$$q = \left(\cos \frac{\theta}{2}, \hat{\mathbf{n}} \sin \frac{\theta}{2} \right). \quad (3.13)$$

La rotación de un punto $\mathbf{p}$ se expresa como $\mathbf{p}' = q \otimes (0, \mathbf{p}) \otimes q^*$, donde q^* es el conjugado. La Figura 3.9 muestra las poses resultantes junto con el cuaternión unitario que las representa: a la izquierda, una composición de roll y pitch; a la derecha, una configuración singular para Euler donde distintas ternas angulares pueden describir la misma orientación física.

Alineación gravitacional con acelerómetro

La IMU de la D435i incluye acelerómetro y giróscopo de tres ejes [4]. En v1, el acelerómetro se usa para estimar la dirección de la gravedad. Esta elección evita integrar el giróscopo o ejecutar un filtro de orientación completo: para detectar suelo alcanza con conocer roll y pitch respecto de la vertical, mientras que yaw no modifica la altura de los puntos ni la coplanaridad del piso. En reposo, en el frame óptico de la cámara, una cámara horizontal mide aproximadamente $\mathbf{a} = (0, -g, 0)$, como se muestra en la Figura 3.10.



(a) Roll y pitch compuestos en un único cuaternión.

(b) Orientación en la singularidad de Euler.

Figura 3.9: Ventajas de la representación mediante cuaterniones para orientaciones compuestas y configuraciones singulares.

Un acelerómetro no mide directamente la aceleración cinemática del sensor, sino la *fuerza específica*: la diferencia entre esa aceleración y la gravedad, expresada en el marco del instrumento. Por eso, aun en reposo entrega una señal de módulo cercano a g y su signo depende de la convención de ejes y del driver. Cuando el sensor además se traslada o vibra, la medición combina la referencia gravitatoria buscada con aceleraciones dinámicas; separar ambas contribuciones a partir de una única muestra no es posible.

A partir de $\mathbf{a} = (a_x, a_y, a_z)^T$ pueden estimarse roll y pitch sin integración temporal:

$$\phi = \text{atan2}(a_y, a_z), \quad \theta = \text{atan2}\left(-a_x, \sqrt{a_y^2 + a_z^2}\right). \quad (3.14)$$

La ecuación 3.14 no estima yaw, ya que la gravedad no contiene información sobre rotación alrededor de la vertical. Estas expresiones permiten interpretar la medición, pero la implementación no calcula ambos ángulos para encadenar sus rotaciones. Para construir el marco alineado a gravedad se calcula la rotación mínima que lleva el vector de aceleración normalizado $\hat{\mathbf{a}}$ al vector canónico $\hat{\mathbf{b}} = (0, -1, 0)$. En forma proporcional:

$$\mathbf{q}_w = 1 + \hat{\mathbf{a}} \cdot \hat{\mathbf{b}}, \quad (3.15)$$

$$\mathbf{q}_v = \hat{\mathbf{a}} \times \hat{\mathbf{b}}. \quad (3.16)$$

Luego se normaliza $\mathbf{q} \leftarrow \mathbf{q}/\|\mathbf{q}\|$. La Figura 3.11 muestra el caso identidad y el caso antiparalelo, donde el eje queda indeterminado y debe resolverse de forma explícita.

Filtrado inercial local para la dirección de gravedad

Durante la marcha el acelerómetro registra aceleraciones lineales de la cabeza, vibraciones y golpes [19], [34]. Por eso, antes de usarlo como referencia vertical se aplica un filtro IIR pasa bajos de primer orden:

$$\mathbf{a}_f[k] = \alpha \mathbf{a}[k] + (1 - \alpha) \mathbf{a}_f[k - 1]. \quad (3.17)$$

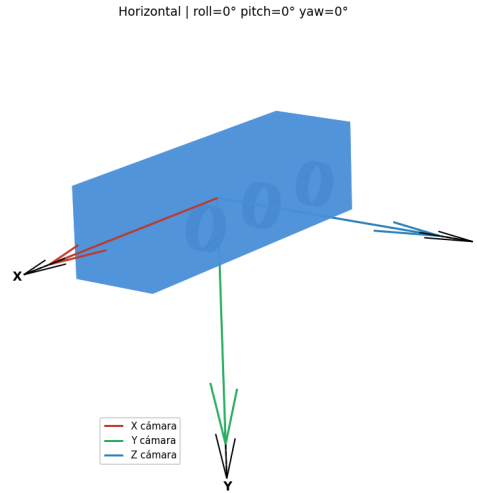


Figura 3.10: Cámara D435i en posición horizontal. El acelerómetro mide $(0, -g, 0)$ en la convención óptica usada por el pipeline.

El parámetro α determina el compromiso entre suavizado y retardo: un valor pequeño atenúa más las variaciones rápidas, pero demora la respuesta ante cambios reales de inclinación [35]. Además del filtrado, la norma de la muestra permite detectar golpes o intervalos en los que la hipótesis de gravedad dominante resulta poco confiable. Los valores y criterios adoptados se especifican en la Sección ??.

3.2.2. Estimación y clasificación geométrica del suelo

Una vez alineada la nube con gravedad, remover el suelo no consiste únicamente en ajustar un plano. El procedimiento debe estimar una tolerancia acorde al ruido, reducir el costo de cálculo, rechazar puntos atípicos, sostener una hipótesis cuando el suelo es poco visible y finalmente clasificar la escena respecto del plano aceptado. La altura de cámara obtenida del mismo modelo actúa además como una comprobación geométrica y como información para las etapas posteriores.

Detección del plano del suelo

Una vez que la nube está alineada a gravedad, el suelo aparece como un conjunto de puntos coplanares a una altura aproximadamente constante. Un plano se representa mediante:

$$\mathbf{n} \cdot \mathbf{p} + d = 0, \quad (3.18)$$

donde $\mathbf{n}$ es la normal unitaria y d la distancia con signo al origen. La distancia con signo de un punto $\mathbf{q}$ al plano es:

$$\delta(\mathbf{q}) = \mathbf{n} \cdot \mathbf{q} + d. \quad (3.19)$$

La ecuación 3.19 permite clasificar puntos según su separación respecto del plano estimado.

Estimación adaptativa de ruido

El ruido de profundidad depende de la distancia y de la reflectividad. Para adaptar la tolerancia de inliers, se estima la distancia media al vecino más cercano sobre un subconjunto

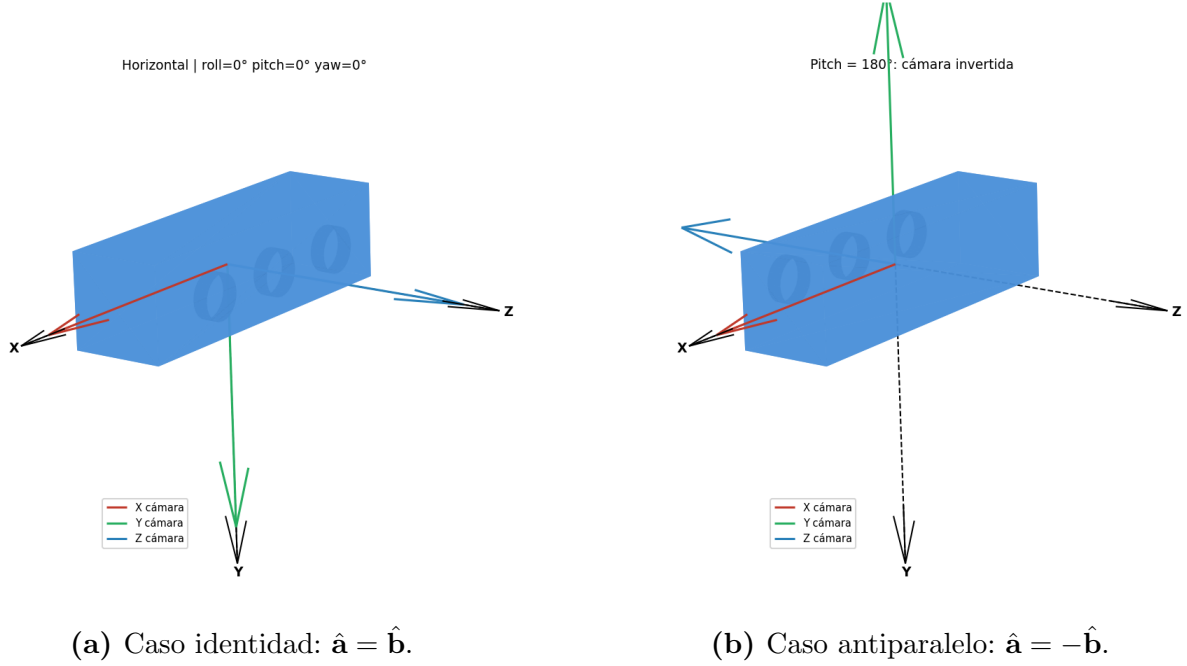


Figura 3.11: Casos límite en el cálculo de la rotación mínima entre el vector de aceleración y la vertical canónica.

de la nube [32]:

$$\bar{d}_{NN} = \frac{1}{m} \sum_{i=1}^m \min_{j \neq i} \|\mathbf{p}_i - \mathbf{p}_j\|. \quad (3.20)$$

La desviación estándar de esa distribución, σ_{NN} , se usa como indicador empírico de la variabilidad espacial de la nube. No equivale estrictamente a la desviación de residuos gaussianos respecto de un plano: también depende de la densidad, las discontinuidades y el submuestreo. Por eso, el factor tres constituye una heurística de adaptación y no un intervalo de confianza probabilístico:

$$\varepsilon = 3 \sigma_{NN}. \quad (3.21)$$

Para evitar tolerancias degeneradas, el método debe limitar el resultado a un intervalo físico plausible y prever un valor de respaldo cuando la estimación no resulte confiable. Los límites adoptados se especifican en la Sección ??.

Submuestreo por voxel grid

Una nube a plena resolución puede contener cientos de miles de puntos. Como el conteo de inliers de RANSAC escala linealmente con la cantidad de puntos, una estrategia habitual de reducción es el submuestreo por *voxel grid*: el espacio se divide en celdas cúbicas de lado s y cada punto $\mathbf{p}_i = (x_i, y_i, z_i)$ se asigna a la clave discreta

$$\mathbf{k}_i = \left(\left\lfloor \frac{x_i}{s} \right\rfloor, \left\lfloor \frac{y_i}{s} \right\rfloor, \left\lfloor \frac{z_i}{s} \right\rfloor \right).$$

Los puntos que caen en una misma celda $V_{\mathbf{k}}$ se reemplazan por un único representante. Una elección habitual consiste en utilizar el promedio aritmético por coordenada,

$$\bar{\mathbf{p}}_{\mathbf{k}} = \frac{1}{N_{\mathbf{k}}} \sum_{\mathbf{p}_i \in V_{\mathbf{k}}} \mathbf{p}_i,$$

es decir, el centroide de las muestras del voxel [17]. Como se esquematiza en la Figure 3.12, solo los puntos contenidos en una celda ocupada aportan al representante de esa celda. No se conserva el mínimo, el máximo ni la profundidad más cercana: el filtro suaviza localmente la nube y reduce la cantidad de puntos. Esta propiedad es apropiada para estimar geometría dominante, como el plano del suelo, pero puede atenuar detalles pequeños si se usara como representación final de obstáculos.

La conveniencia de aplicar esta reducción depende del objetivo: es útil para estimar geometría dominante, pero no necesariamente para conservar obstáculos pequeños. La alternativa escogida para el recorrido activo se explicita en la Sección ??.

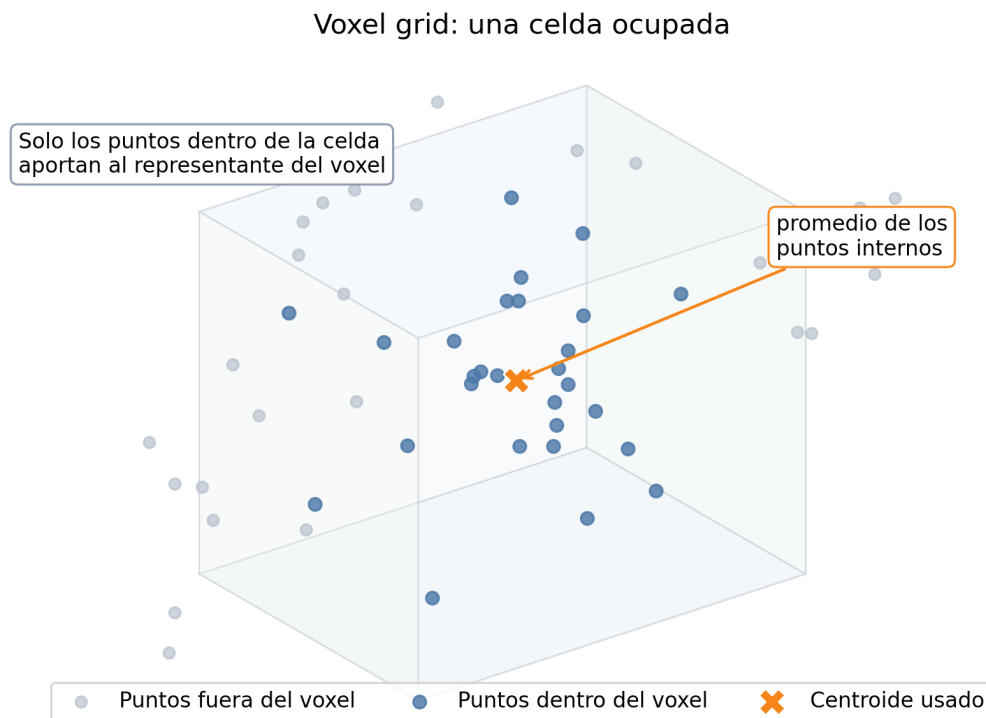


Figura 3.12: Esquema tridimensional de submuestreo por *voxel grid*. Los puntos originales dentro de cada celda ocupada se reemplazan por un único representante ubicado en el centroide de las muestras. Elaboración propia.

RANSAC y refinamiento por mínimos cuadrados

En una escena real, el plano del suelo coexiste con obstáculos, paredes y ruido. RANSAC (*Random Sample Consensus*) estima modelos paramétricos en presencia de valores atípicos [12]. Para planos, cada iteración muestrea tres puntos, ajusta un candidato y cuenta inliers con distancia menor que ε . Si la fracción de inliers es ρ , la probabilidad de fallar en K iteraciones es:

$$P_{\text{fail}} = (1 - \rho^3)^K. \quad (3.22)$$

Esta relación permite controlar el compromiso entre robustez y tiempo de cómputo.

El modelo no requiere que el suelo coincida con un valor constante exacto de altura. Si el piso presenta una pendiente moderada, el conjunto de puntos puede seguir explicándose mediante un plano inclinado; la condición relevante para RANSAC es la distancia de cada muestra al plano candidato, no la igualdad de una coordenada particular. La Figure 3.13 ilustra esta idea sobre un corte bidimensional, usando la convención geométrica adoptada en el sistema.

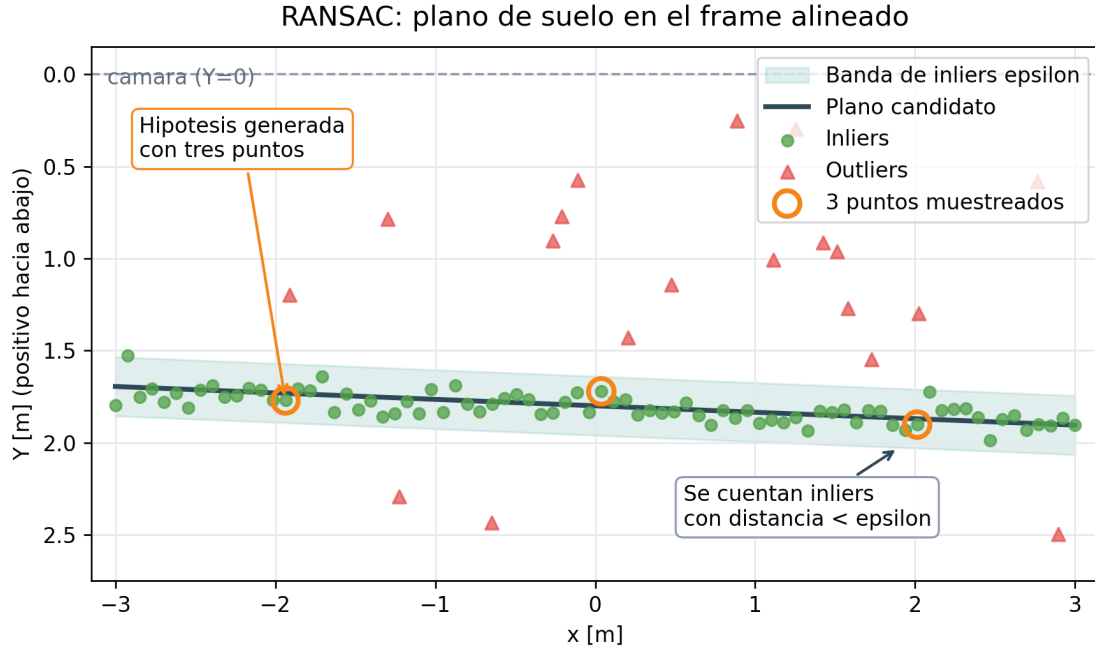


Figura 3.13: Esquema bidimensional de RANSAC para ajuste de planos. Cada hipótesis se genera a partir de tres puntos y luego se cuentan como *inliers* las muestras cuya distancia al plano candidato es menor que la tolerancia ε . Elaboración propia.

Luego de seleccionar los inliers, el plano se refina por mínimos cuadrados. Se minimiza:

$$J(\mathbf{n}, d) = \sum_{i=1}^n (\mathbf{n}^T \mathbf{p}_i + d)^2. \quad (3.23)$$

Derivando respecto de d se obtiene el centroide:

$$\bar{\mathbf{p}} = \frac{1}{n} \sum_{i=1}^n \mathbf{p}_i, \quad d^* = -\mathbf{n}^T \bar{\mathbf{p}}. \quad (3.24)$$

El problema se reduce a hallar la dirección de menor varianza de la matriz de covarianza:

$$C = \frac{1}{n} \sum_{i=1}^n (\mathbf{p}_i - \bar{\mathbf{p}})(\mathbf{p}_i - \bar{\mathbf{p}})^T. \quad (3.25)$$

La normal óptima es el autovector asociado al menor autovalor [26, App. A4]:

$$\mathbf{n}^* = \mathbf{q}_3, \quad d^* = -\mathbf{n}^{*T} \bar{\mathbf{p}}. \quad (3.26)$$

Para matrices 3×3 simétricas, el cálculo puede realizarse mediante iteraciones de Jacobi. Cada iteración aplica una rotación ortogonal $R_{ij}(\theta)$:

$$C_{k+1} = R_{ij}(\theta)^T C_k R_{ij}(\theta), \quad (3.27)$$

con:

$$\theta = \frac{1}{2} \text{atan2}(2(C_k)_{ij}, (C_k)_{ii} - (C_k)_{jj}). \quad (3.28)$$

Robustez ante suelo parcialmente visible

Cuando la cámara apunta al frente, el suelo puede ocupar una fracción pequeña de la nube. Una métrica de calidad respecto del total de puntos penalizaría escenas donde el piso es visible solo en una banda inferior. Para resolverlo se restringe el muestreo a una banda candidata:

$$\mathcal{B} = \{i : Y^* - \delta_l \leq Y_i \leq Y^* + \delta_h\}, \quad (3.29)$$

donde Y^* se obtiene mediante un histograma de alturas. La calidad se calcula en relación con esa banda:

$$\rho = \frac{|\text{inliers} \cap \mathcal{B}|}{|\mathcal{B}|}. \quad (3.30)$$

Además, como el error axial del D435 crece con la distancia [25], la tolerancia puede variar por punto:

$$\varepsilon_i = \max(\varepsilon, k\|\mathbf{p}_i\|). \quad (3.31)$$

Con este criterio, el plano no se estima a partir de toda la nube por igual. La búsqueda se concentra en la región donde es esperable observar el piso y evalúa la calidad dentro de esa región, incorporando además una tolerancia compatible con el ruido de profundidad.

Estos recursos proporcionan los fundamentos para construir un estimador robusto, pero su orden y sus condiciones de aceptación dependen del método adoptado. El procedimiento utilizado, incluidos la reutilización temporal, los caminos de fallo y la clasificación final, se presenta en la Sección ??.

Clasificación geométrica y región superior excluida

Una vez obtenido el plano, la distancia con signo permite separar los puntos compatibles con el suelo de los obstáculos. Se etiqueta como suelo un punto dentro de la tolerancia o situado por debajo del plano estimado. Entre los restantes, los puntos cuya coordenada satisface $Y < -\Delta_c$ se asignan a una región superior excluida; los demás se conservan como obstáculos. El valor adoptado para Δ_c se especifica en la Sección ??.

Esta región no constituye una detección semántica de techos. Reúne cualquier medición situada más de Δ_c por encima de la cámara y puede contener ramas, carteles u otros elementos elevados. Se mantiene el término **CEILING** únicamente al referirse a la etiqueta del mensaje o del código. Su descarte y el empleo submuestreado de esos puntos como extremos de espacio libre son decisiones que deben validarse experimentalmente, porque una proyección 2D puede perder información sobre obstáculos elevados.

La aplicación de esta clasificación sobre una escena real se muestra en las Figuras ?? y ??. La primera permite observar el cambio de marco previo a la estimación; la segunda muestra la separación resultante entre suelo, región superior excluida y los puntos conservados como obstáculos.

Realimentación de la altura de cámara

La distancia del origen al plano estimado proporciona una medición de la altura de la cámara. Para impedir que errores aislados modifiquen bruscamente la grilla, esta estimación solo se acepta cuando el plano supera requisitos de calidad, cantidad de inliers e inclinación. La altura aceptada se filtra, se limita el cambio por actualización y los saltos grandes deben repetirse de forma coherente antes de iniciar una transición más rápida.

El valor resultante actualiza el límite vertical superior usado por la codificación espacial. De este modo, las cinco filas mantienen una relación con el espacio comprendido entre la zona

inferior y la altura observada de la cámara, aun cuando cambie el usuario o la posición de montaje. Esta realimentación modifica la asignación vertical, pero no altera la matriz final de 5 filas por 10 columnas.

El procedimiento adoptado para validar y realimentar esta altura se presenta en la Sección ??; su evaluación experimental se define en el bloque v1 del capítulo de experimentos.

3.3. Fundamentos de la versión v2: odometría, mapa local y conciencia espacial

La versión v2 parte de la percepción filtrada alcanzada en v1 y agrega memoria espacial para cubrir su principal limitación restante. Para que un obstáculo siga influyendo cuando abandona el campo frontal, las observaciones locales deben transformarse a un marco común, actualizar una grilla de ocupación y evaluarse junto con la velocidad del usuario. En la configuración final, esta etapa se apoya en `imu_filter_madgwick`, `rtabmap_odom`, `local_mapper` y `spatial_awareness`.

Esta incorporación requiere distinguir el movimiento incremental usado para acumular mediciones del problema más amplio de construir y corregir un mapa global.

3.3.1. Estimación de pose y coherencia temporal

Acumular observaciones exige conocer no solo cuánto se movió la cámara, sino también a qué instante corresponde cada pose y en qué convención están expresados sus ejes. Por eso el bloque distingue primero odometría de localización y SLAM, y luego reúne las tres condiciones de la pose usada por el mapa: orientación inercial filtrada, desplazamiento RGB-D y asociación temporal de ambas mediciones dentro de marcos compatibles.

Odometría, localización, mapeo y SLAM

Estos términos describen problemas relacionados pero no equivalentes. La *odometría* estima el cambio de pose a partir del movimiento observado y, por ello, su error tiende a acumularse. La *localización* estima la pose respecto de una referencia conocida. El *mapeo* construye una representación del entorno suponiendo disponible una estimación de pose. SLAM resuelve conjuntamente localización y mapeo, normalmente incorporando mecanismos para corregir deriva al reconocer lugares ya visitados [12].

En v2 no se construye un mapa SLAM global. La odometría RGB-D se utiliza para expresar mediciones sucesivas en un marco común y mantener una memoria espacial acotada. Esta distinción permite interpretar correctamente el papel de RTAB-Map: se emplea su componente odométrico, no su sistema completo de cierre de lazo y optimización global.

Filtro de Madgwick para orientación IMU

La incorporación de Madgwick corresponde a v2, no a v1. En v1 la señal inercial relevante es el acelerómetro, porque permite estimar la vertical local a partir de la gravedad con bajo costo. En v2, en cambio, la odometría necesita una orientación IMU filtrada que combine acelerómetro y giroscopo. Para esta versión, el controlador de la D435i se configura con `unite_imu_method=1`: además de los flujos inerciales separados, publica el tópico unificado `/camera/camera/imu` que consume Madgwick. La orientación de ese mensaje no constituye por sí misma una estimación filtrada utilizable por la odometría; Madgwick publica el cuaternión resultante en `/imu/data_filtered`.

El problema es que ninguna de las dos mediciones inerciales resuelve por sí sola la orientación de forma robusta. El giróscopo describe bien los cambios rápidos de orientación, pero su integración acumula deriva. El acelerómetro, en cambio, ofrece una referencia de baja frecuencia asociada a la gravedad, útil para corregir roll y pitch cuando la aceleración dinámica no domina la medición [19]. Esta complementariedad motiva el uso de un filtro que propague la orientación con el giróscopo y la corrija con la dirección de gravedad medida por el acelerómetro.

La deriva resulta especialmente relevante para el yaw. En una aproximación discreta, cuando el eje vertical del sensor coincide con la vertical del marco de referencia, la integración de la velocidad angular alrededor de ese eje puede expresarse como

$$\psi(t) = \psi(t - \Delta t) + \omega_y(t) \Delta t. \quad (3.32)$$

La ecuación 3.32 permite observar que un sesgo constante del giróscopo produce un error angular acumulativo [19]. La corrección gravitatoria limita este efecto en roll y pitch, pero no proporciona una referencia absoluta para el yaw, porque una rotación alrededor de la vertical no modifica la dirección de la gravedad medida por el acelerómetro.

Para combinar ambas mediciones se usa el filtro de Madgwick [24]. Para mantener la correspondencia con el diagrama original, se adopta en esta sección su notación: S es el marco del sensor, E es el marco de referencia, el sombrero indica un cuaternión unitario y t identifica la muestra actual. La rama inferior de la Figura 3.14 comienza con la derivada de orientación calculada a partir del giróscopo:

$${}^S_E \dot{q}_{\omega,t} = \frac{1}{2} {}^S_E \hat{q}_{\text{est},t-1} \otimes {}^S \omega_t, \quad (3.33)$$

donde ${}^S \omega_t$ es la velocidad angular expresada en el marco del sensor y ${}^S_E \hat{q}_{\text{est},t-1}$ es la orientación estimada en la muestra anterior. En la rama superior, el acelerómetro normalizado ${}^S \hat{a}_t$ define la dirección de gravedad observada. La diferencia respecto de la gravedad predicha por el cuaternión anterior se expresa mediante la función objetivo f_g ; su jacobiano J_g permite obtener la dirección de corrección:

$$\nabla f_g = J_g^T ({}^S_E \hat{q}_{\text{est},t-1}) f_g ({}^S_E \hat{q}_{\text{est},t-1}, {}^S \hat{a}_t). \quad (3.34)$$

El sumador central combina ambas ramas restando el gradiente normalizado, ponderado por la ganancia β :

$${}^S_E \dot{q}_{\text{est},t} = {}^S_E \dot{q}_{\omega,t} - \beta \frac{\nabla f_g}{\|\nabla f_g\|}. \quad (3.35)$$

Finalmente, el bloque de integración actualiza la orientación, el bloque siguiente la normaliza y los dos retardos z^{-1} del diagrama realimentan el valor correspondiente a la muestra anterior:

$${}^S_E \hat{q}_{\text{est},t} = \text{norm} ({}^S_E \hat{q}_{\text{est},t-1} + {}^S_E \dot{q}_{\text{est},t} \Delta t). \quad (3.36)$$

En esta sección se presentan únicamente las relaciones necesarias para interpretar el diagrama y comprender el uso del filtro dentro del sistema. La derivación completa de la función objetivo, su jacobiano, el descenso de gradiente y las expresiones optimizadas para las implementaciones IMU y MARG puede consultarse en el informe interno original de Madgwick [36].

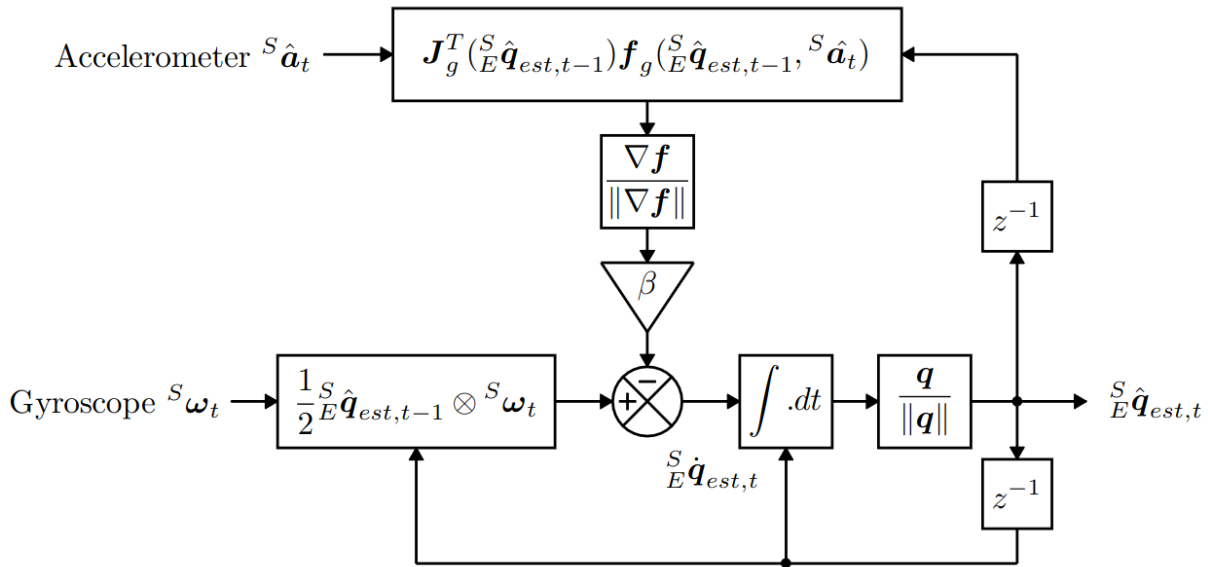


Figura 3.14: Diagrama de bloques del filtro de Madgwick para una implementación IMU sin magnetómetro. La estimación propagada mediante el giroscopo se corrige por descenso de gradiente a partir de la dirección de gravedad observada por el acelerómetro, y luego se integra y normaliza. Fuente: Madgwick [36, Fig. 2].

Sin magnetómetro, la gravedad no hace observable el yaw absoluto. Por eso, en este sistema la corrección de la rama superior limita principalmente la deriva de roll y pitch, mientras que la evolución horizontal debe provenir de las correspondencias visuales. La implementación produce el cuaternión de `/imu/data_filtered`, utilizado para inicializar y regular `rgbd_odometry`; este acoplamiento no constituye una optimización visual-inercial conjunta. La elección de Madgwick responde, además, a su bajo costo de cálculo por muestra, adecuado para ejecución embebida.

Odometría RGB-D con RTAB-Map

La odometría visual estima el movimiento de la cámara a partir de cambios entre fotogramas consecutivos [26]. En v2 se utiliza `rgbd_odometry` de RTAB-Map [23]. El nodo consume imagen RGB, parámetros intrínsecos, profundidad alineada a color y la IMU filtrada. Conceptualmente, detecta rasgos visuales, los asocia entre cuadros y usa la profundidad para convertir sus píxeles en correspondencias métricas 3D. La transformación rígida que mejor explica esas correspondencias produce un incremento de pose. Las asociaciones incorrectas deben rechazarse como valores atípicos; la falta de textura, las oclusiones y el desenfoque por movimiento pueden reducir o interrumpir el seguimiento.

RTAB-Map puede operar como biblioteca de SLAM y mapeo, pero en este sistema se usa su odometría RGB-D como fuente local de pose. La salida se publica como `nav_msgs/Odometry` y se remapea a `nav_odom`. El marco `odom` no representa un sistema global absoluto; es un marco local continuo, con origen definido al inicio de la ejecución y con deriva acumulada a largo plazo. Esta propiedad es suficiente para un mapa local de corto alcance, porque la coherencia requerida abarca segundos y metros recientes, no un mapa permanente de todo lo recorrido previamente.

La secuencia concreta utilizada para documentar esta operación debe obtenerse de datos reales; su incorporación se plantea en la Sección ?? como evidencia del método adoptado, no como fundamento general de la odometría visual.

Marcos de referencia y convención REP-103

ROS define convenciones estándar de unidades y marcos en REP-103 [37]. Sin embargo, los flujos ópticos de cámara utilizan una convención natural para imágenes: x hacia la derecha, y hacia abajo y z hacia adelante. El sistema combina ambas perspectivas: RTAB-Map publica odometría en sus marcos ROS, mientras que el pipeline geométrico interno trabaja con la convención óptica de la cámara.

La transformación entre convenciones no altera la información física; solo expresa la misma pose en ejes compatibles con la grilla local. Para el mapa 2D se conserva la posición horizontal del sensor y el yaw extraído de la orientación. Roll y pitch no se aplican en esta etapa porque la nube ya fue alineada con gravedad en v1. Esta separación evita mezclar la estabilización vertical de la percepción con la acumulación horizontal del mapa.

Para la fuente RTAB-Map, la conversión de la traslación publicada a las coordenadas horizontales internas es:

$$x_{\text{int}} = -y_{\text{ROS}}, \quad z_{\text{int}} = x_{\text{ROS}}. \quad (3.37)$$

La orientación se precompone con el cuaternión $q_{\text{ROS} \rightarrow \text{int}} = (0,5, 0,5, -0,5, 0,5)$, escrito como (w, x, y, z) , y después se proyecta la dirección frontal sobre el plano XZ para conservar únicamente yaw. Cada punto local $\mathbf{p}_l = (x, 0, z)$ se inserta mediante

$$\mathbf{p}_{\text{odom}} = q_{\text{yaw}} \mathbf{p}_l q_{\text{yaw}}^* + (x_{\text{int}}, 0, z_{\text{int}}). \quad (3.38)$$

Estas expresiones fijan tanto el orden del cuaternión como el sentido de la transformación y evitan interpretar un cambio de base como movimiento físico adicional.

La verificación de estas convenciones contra los marcos TF publicados forma parte del método de integración de v2 y se ubica en la Sección ??, donde puede contrastarse con RViz sin convertir una convención específica del sistema en una propiedad general de ROS.

Sincronización temporal de observaciones y odometría

El mapa local combina observaciones y poses producidas a frecuencias distintas. Asociar datos solo por orden de llegada puede aplicar una pose incorrecta a una nube y desplazar artificialmente los obstáculos. La sincronización exacta exige sellos iguales; la aproximada acepta una diferencia acotada; la selección de la última pose anterior mantiene causalidad pero introduce retardo; y la interpolación estima una pose al instante de la observación a partir de muestras vecinas.

En primera aproximación, un desfase Δt durante un movimiento con velocidad lineal v y angular ω puede introducir errores del orden de $v\Delta t$ en posición y $\omega\Delta t$ en orientación. Por ello, toda política debe declarar la antigüedad admisible y su conducta durante el arranque. La selección concreta se muestra en la Sección ??.

3.3.2. Representación y actualización del mapa local

La memoria espacial de v2 se construye como una ventana local de ocupación, no como un mapa permanente del entorno. Las subsecciones siguientes justifican esa representación y describen las operaciones que le dan significado: acumular evidencia probabilística, marcar espacio libre a lo largo de cada rayo, desplazar la ventana con el usuario y limitar la persistencia de observaciones antiguas. En conjunto, estas operaciones determinan qué recuerda el sistema y durante cuánto tiempo esa memoria continúa siendo confiable.

Representaciones espaciales locales

Se distingue entre mapas globales, que buscan representar un entorno completo, y mapas locales de corto alcance, centrados en el usuario y limitados a una ventana espacial. Los mapas locales permiten actualizaciones frecuentes con baja latencia, por lo que son adecuados para la evitación reactiva de obstáculos[15], [18]. En esta tesis se adopta una grilla de ocupación 2D local en el marco `odom`.

La grilla local se expresa en el plano horizontal. La altura estimada de la cámara respecto del suelo, publicada por el filtro geométrico, se usa para ubicar correctamente el origen de la visualización y mantener consistencia entre la nube 3D, el plano de suelo y la representación 2D.

Grillas de ocupación y log-odds

La grilla de ocupación fue introducida por Elfes [11] como una discretización probabilística del espacio. Cada celda m_i almacena la probabilidad de ocupación condicionada al historial de observaciones $z_{1:t}$. Antes de observarla se parte de una probabilidad *a priori*; cada medición aporta evidencia y produce una probabilidad *posterior*. El modelo directo $P(z_t | m_i)$ describe qué medición sería esperable dado el estado de la celda. Para construir el mapa se utiliza, en cambio, un *modelo inverso del sensor* $P(m_i | z_t)$: estima si una celda debe considerarse ocupada o libre dada la medición. En un sensor de alcance, normalmente el espacio recorrido hasta un impacto aporta evidencia libre y el extremo medido aporta evidencia ocupada.

Aplicar directamente la regla de Bayes en cada actualización requiere productos y normalizaciones. Para expresarla como sumas se usa la escala de *log-odds*:

$$l_t(m_i) = l_{t-1}(m_i) + \underbrace{\log \frac{P(m_i = 1 | z_t)}{P(m_i = 0 | z_t)}}_{l_{\text{sensor}}(z_t, m_i)} - l_0. \quad (3.39)$$

La ecuación 3.39 convierte la actualización bayesiana recursiva en una suma [12]. Las celdas impactadas por obstáculos reciben un incremento positivo l_{occ} , mientras que las celdas atravesadas por rayos libres reciben un incremento negativo l_{free} . Para estabilidad numérica, l_t se satura en $[l_{\text{mín}}, l_{\text{máx}}]$.

Los incrementos dependen de las probabilidades asignadas por el modelo inverso del sensor:

$$l_{\text{occ}} = \log \frac{p_{\text{occ}}}{1 - p_{\text{occ}}}, \quad l_{\text{free}} = \log \frac{p_{\text{free}}}{1 - p_{\text{free}}}. \quad (3.40)$$

Estas probabilidades expresan cuánto se confía en que una celda esté ocupada después de un impacto o después de ser atravesada por un rayo libre. No son características nominales de la cámara ni valores fijados por la formulación de Elfes: deben seleccionarse para el sensor, el procesamiento previo y el comportamiento temporal buscado.

La probabilidad puede recuperarse como:

$$P(m_i = 1 | z_{1:t}) = 1 - \frac{1}{1 + \exp(l_t(m_i))}. \quad (3.41)$$

Esta forma permite publicar la grilla en `nav_msgs/OccupancyGrid`. La resolución, la extensión de la ventana, los incrementos y los límites de saturación son decisiones del método: determinan respectivamente detalle espacial, memoria disponible, confianza asignada a cada observación y cantidad máxima de evidencia acumulable. Los valores adoptados y su calibración se presentan en las Secciones ?? y ??.

El valor de log-odds es la representación interna del mapeador. Una celda jamás observada debe distinguirse de otra observada cuya evidencia se encuentra equilibrada: ambas pueden tener probabilidad 0,5, pero solo la segunda contiene información. La codificación concreta en `nav_msgs/OccupancyGrid` se describe en la Sección ??.

La extensión de la ventana no implica que exista una medición en toda esa superficie. La cámara aporta observaciones dentro de su campo de visión y alcance; la zona posterior y las zonas laterales conservan únicamente información observada previamente mientras permanezcan dentro del radio local. Las regiones todavía no observadas continúan en estado desconocido.

Por ejemplo, una observación ocupada lleva una celda desconocida a $l = 2,197$, equivalente a $P \simeq 0,90$ y a `data[i] = 90`. Una observación libre posterior la reduce a $l = 1,350$, equivalente a $P \simeq 0,79$ y `data[i] = 79`. Las observaciones libres siguientes producen sucesivamente 62 y 41. Se necesitan, por lo tanto, observaciones contradictorias repetidas para invertir la evidencia. Este ejemplo muestra tanto la memoria temporal interna como su codificación en el mensaje publicado.

La Figura 3.15 representa la secuencia numérica anterior. En cada punto se muestran por separado el log-odds del eje vertical, la probabilidad recuperada y el valor almacenado en `data[i]` del mensaje `OccupancyGrid`. El cruce por $l = 0$ permite identificar cuándo la evidencia acumulada deja de favorecer ocupación y pasa a favorecer espacio libre.

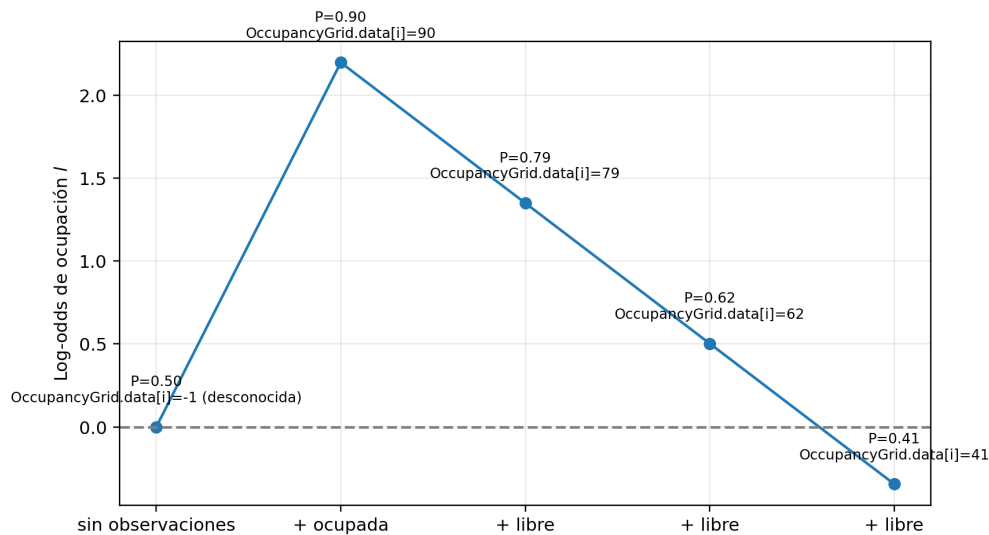


Figura 3.15: Evolución del log-odds de una celda inicialmente desconocida y conversión de cada estado al valor publicado en `OccupancyGrid.data[i]`, utilizando la configuración inicial previa a la calibración.

Ray casting con Bresenham

Además de marcar ocupada la celda del obstáculo, el mapa debe marcar como libre el espacio atravesado por el rayo de medición. El segmento continuo medido por la cámara debe convertirse, por lo tanto, en una sucesión de celdas de la grilla. Para ello se usa el algoritmo de Bresenham [16]. Dado un origen (x_0, z_0) y un destino (x_1, z_1) en coordenadas enteras de celda, el algoritmo avanza sobre el eje dominante y decide cuándo incrementar el eje subordinado mediante una variable de error acumulado. El resultado es una aproximación discreta del segmento formada por celdas contiguas; no es el conjunto geométrico de todas las celdas que la recta llega a tocar. Esta construcción evita calcular intersecciones en punto flotante.

En este sistema se aplican dos tipos de rayos. Los rayos con obstáculo marcan libres las celdas intermedias y ocupada la celda final. Los rayos asociados a suelo, región superior excluida o mediciones sin obstáculo dentro del rango marcan libre también el extremo final. Para no borrar evidencia acumulada de forma excesiva, una celda recibe evidencia libre como máximo una vez por frame.

La Figura 3.16 muestra la discretización espacial de ambos casos. La línea azul une, en orden, los centros de las celdas seleccionadas por Bresenham desde la cámara hasta el destino; por eso no debe interpretarse como la intersección geométrica exacta entre una recta y la grilla. Las celdas verdes reciben evidencia libre. Si el extremo corresponde a un obstáculo, la última celda se reserva para evidencia ocupada (rojo). Si la medición no contiene un obstáculo dentro del rango, el recorrido se extiende hasta el límite medible representado y su extremo también recibe evidencia libre.

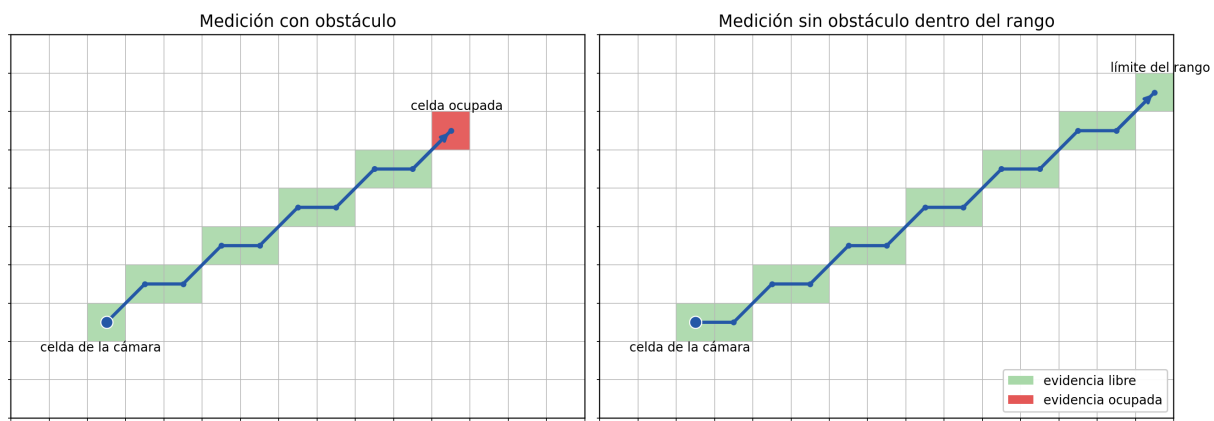


Figura 3.16: Recorrido discreto de Bresenham sobre la grilla: las celdas intermedias reciben evidencia libre; ante un obstáculo el extremo es ocupado y, en caso contrario, la evidencia libre se extiende hasta el límite del rango representado.

Ventana local deslizante y olvido espacial

La memoria espacial se mantiene en una ventana finita. Cuando la odometría indica desplazamiento del sensor, la grilla se desplaza una cantidad entera de celdas y las celdas nuevas se reinician con prior neutro. Para evitar pérdida de resolución a bajas velocidades, los desplazamientos sub-celda se acumulan hasta superar el tamaño de celda.

Además del corrimiento de ventana, se emplea un radio de olvido local: las celdas demasiado alejadas del sensor vuelven al prior. Este mecanismo impide que observaciones viejas dominen indefinidamente y mantiene el mapa enfocado en la región útil para navegación inmediata.

La ventana está centrada alrededor de la posición estimada del usuario, pero sus ejes permanecen alineados con *odom*: no gira con cada cambio de yaw. La traslación desplaza los contenidos por una cantidad entera de celdas y un acumulador conserva las fracciones menores que 0,10 m hasta completar una celda. Las posiciones de los obstáculos se rotan y trasladan al marco común antes de actualizar la grilla.

El corrimiento entero, la acumulación subcelda y la conservación de los ejes de *odom* son decisiones del método de actualización adoptado; su diagrama se presenta en la Sección ??.

Filtrado espacial y temporal

El filtrado espacial aparece en dos niveles. Primero, la reducción a *DepthGrid* resume múltiples muestras en celdas. Segundo, la grilla de ocupación acumula evidencia por celda en el

plano local. El filtrado temporal surge de la actualización log-odds: observaciones persistentes refuerzan ocupación y mediciones libres repetidas la reducen. Este comportamiento suaviza detecciones esporádicas, aunque requiere saturación y olvido para evitar que el mapa se vuelva rígido frente a cambios reales. El olvido puede implementarse por decaimiento temporal, por evidencia contradictoria o eliminando regiones que abandonan la ventana local; la política adoptada se especifica en la Sección ??.

3.3.3. Evaluación y comunicación del riesgo contextual

El mapa local no se transmite directamente al usuario: primero debe reducirse a una indicación de riesgo compatible con la salida frontal ya existente. Este bloque describe esa reducción en dos pasos. La capa de conciencia espacial selecciona obstáculos relevantes fuera del campo frontal y pondera su urgencia mediante el tiempo hasta colisión; la fusión háptica incorpora ese resultado sin sustituir la percepción instantánea de v_l ni generar una segunda interfaz de salida.

Conciencia espacial y tiempo hasta colisión

La capa de conciencia espacial evalúa riesgos fuera del campo frontal ya comunicado por la grilla háptica. Para no interpretar ruido aislado como un objeto, las celdas suficientemente ocupadas pueden agruparse en componentes conexas y descartarse los grupos sin soporte espacial. Esta operación es una heurística: mejora robustez ante activaciones aisladas, pero puede omitir obstáculos delgados, de modo que sus umbrales requieren validación.

Para cada celda candidata se calcula su posición relativa respecto de la pose del usuario y se descarta si cae dentro de la apertura frontal observable, región ya atendida por la percepción frontal. Luego se evalúa si intersecta una zona compatible con colisión, orientada según el vector de velocidad y no necesariamente según el frente observable. Su semiancho debe considerar el radio corporal r_{cuerpo} y la mitad de la diagonal de una celda cuadrada de lado s :

$$r_c = r_{\text{cuerpo}} + \frac{s}{\sqrt{2}}. \quad (3.42)$$

Esta tolerancia incluye celdas cuyo centro queda ligeramente fuera del radio corporal, pero cuya extensión todavía puede intersectar la trayectoria. Finalmente se exige velocidad de cierre suficiente. La velocidad de cierre se calcula como

$$v_c = \max \left(0, \mathbf{v} \cdot \frac{\mathbf{p}_o - \mathbf{p}_u}{\|\mathbf{p}_o - \mathbf{p}_u\|} \right), \quad (3.43)$$

donde $\mathbf{p}_o$ y $\mathbf{p}_u$ son las posiciones del obstáculo y del usuario. Para que el horizonte represente la llegada del obstáculo al contorno corporal y no al centro de la pose, la separación empleada es

$$d = \max (0, \|\mathbf{p}_o - \mathbf{p}_u\| - r_{\text{cuerpo}}). \quad (3.44)$$

El tiempo hasta colisión se define entonces como:

$$t_{tc} = \frac{d}{v_c}, \quad (3.45)$$

donde d es la separación entre el contorno corporal y el obstáculo, mientras que v_c es la componente de velocidad dirigida hacia él. El modelo supone obstáculo estacionario y velocidad del usuario constante durante el horizonte analizado; no predice el movimiento de terceros. La

ecuación 3.45 prioriza urgencia dinámica, no solo proximidad geométrica. La intensidad asociada puede escalarse linealmente entre un tiempo mínimo y un horizonte máximo:

$$I_{\text{riesgo}} = 100 \text{ sat}_{[0,1]} \left(\frac{t_{\text{máx}} - t_{\text{tc}}}{t_{\text{máx}} - t_{\text{mín}}} \right). \quad (3.46)$$

Los límites temporales, las regiones semánticas, la histéresis, la vigencia de los datos y la selección de una fuente de velocidad son decisiones del método. Deben explicitarse porque el TTC depende linealmente de la velocidad elegida; se detallan en la Sección ??.

La Figura 3.17 reúne estas relaciones en una vista superior. El vector $\mathbf{v}$ determina el eje de la zona compatible con colisión; v_c es su proyección sobre la dirección del obstáculo. La distancia d se mide desde el contorno corporal y no desde el centro de la pose. Los valores numéricos constituyen un ejemplo geométrico reproducible, no un resultado experimental.

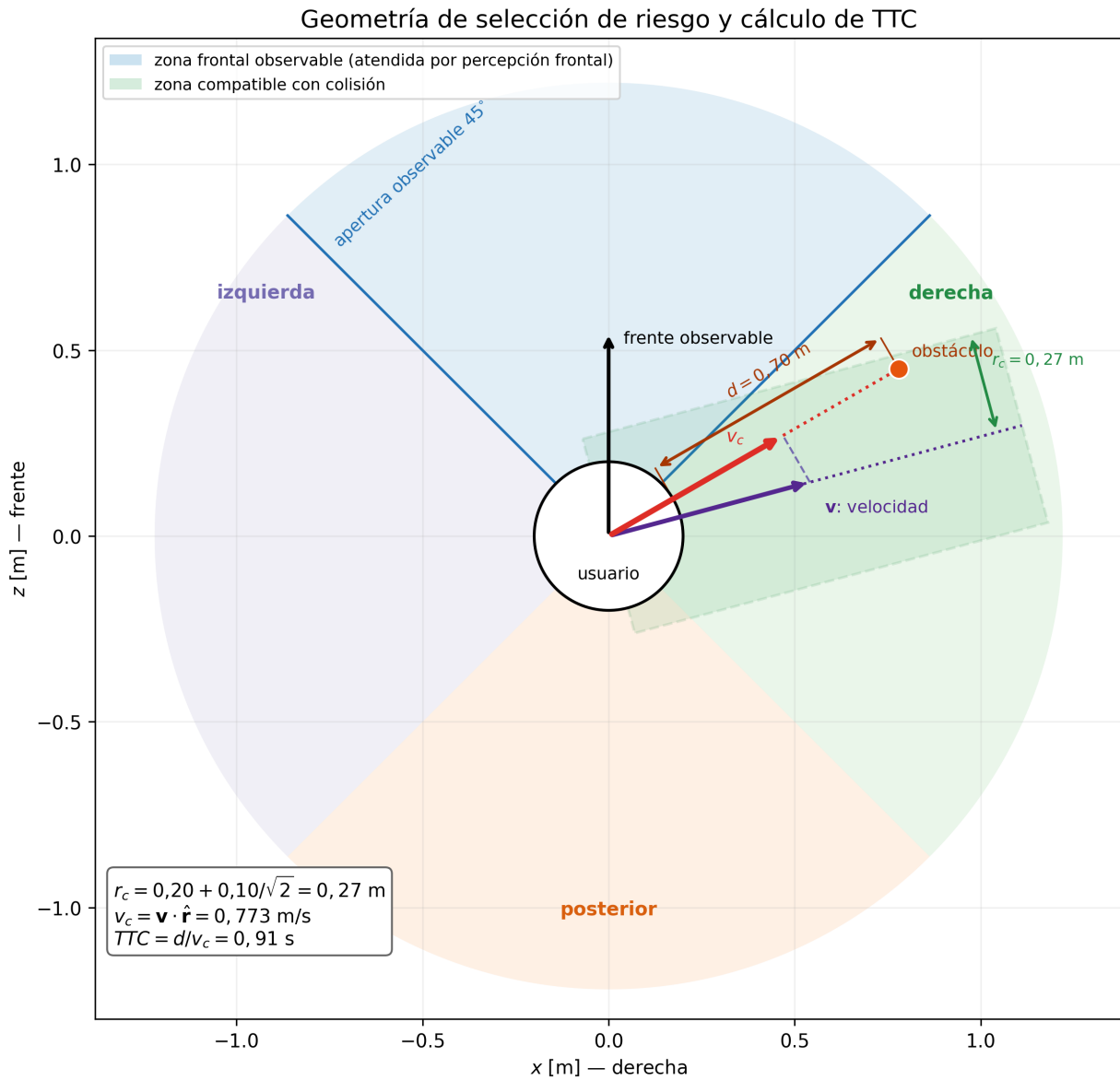


Figura 3.17: Geometría empleada para seleccionar un riesgo contextual y calcular el tiempo hasta colisión. La zona frontal observable queda a cargo de la percepción frontal; la evaluación contextual considera las regiones laterales y posterior.

Fusión háptica con riesgo contextual

En v2, la salida háptica ya no representa únicamente la percepción frontal instantánea. La grilla frontal sigue codificando distancia a obstáculos en el campo visual, mientras que la conciencia espacial aporta señales semánticas de riesgo lateral y posterior. La fusión se realiza por máximo de intensidad: si una celda ya está activa por percepción frontal, una señal contextual solo la aumenta si su intensidad es mayor.

La asignación de cada región a columnas concretas y la retención breve de la última advertencia son decisiones del método descriptas en la Sección ??.

Esta regla conserva el mayor valor numérico sin promediar señales, pero no demuestra por sí sola que se conserve el riesgo físicamente más urgente: la intensidad frontal deriva de distancia y la contextual de TTC. Su comparación presupone que ambas escalas 0–100 fueron calibradas de manera compatible. Asimismo, la codificación posterior en columnas centrales es una convención que debe verificarse con usuarios y con el dispositivo físico del proyecto.

La obtención de las tres matrices reproducibles corresponde a la validación funcional de v2 y se especifica en el bloque experimental de memoria espacial y riesgo contextual. Aquí se mantiene únicamente la definición de la operación de fusión y la advertencia sobre la compatibilidad de escalas.

3.3.4. Métricas de evaluación para percepción y mapeo

Una evaluación de clasificación compara cada predicción con una referencia. Un verdadero positivo (TP) es una ocupación detectada que existe en la referencia; un falso positivo (FP) es una ocupación inexistente; y un falso negativo (FN), una ocupación real omitida. A partir de estos conteos:

$$\text{precision} = \frac{TP}{TP + FP}, \quad \text{recall} = \frac{TP}{TP + FN}, \quad F_1 = 2 \frac{\text{precision recall}}{\text{precision} + \text{recall}}. \quad (3.47)$$

La precisión cuantifica qué proporción de las detecciones fue correcta; el *recall*, qué proporción de los obstáculos existentes fue detectada; y F_1 resume ambas magnitudes. La latencia *end-to-end*, la tasa de publicación, la estabilidad temporal y el consumo de recursos también requieren extremos, ventanas y condiciones explícitas. Su selección por versión, las referencias y los procedimientos concretos se establecen en el capítulo experimental.

Capítulo 4

Arquitectura del sistema

4.1. Arquitectura general

El sistema se compone de un conjunto de nodos ROS 2 que se ejecutan sobre una Raspberry Pi 5, orquestados por el archivo de lanzamiento `nav.launch.py` del paquete `nav_bringup`. Todos los nodos de procesamiento quedan contenidos en la misma unidad de cómputo y se comunican exclusivamente a través del middleware DDS (`rmw_cyclonedds_cpp`). La frontera de la implementación física se ubica en la transmisión UART: el sistema genera y serializa los comandos de intensidad, pero no incluye el receptor, la electrónica de potencia ni los actuadores. Distintos nodos de visualización del paquete (`output_viewer`) pueden ejecutarse opcionalmente en una PC separada, suscribiéndose a los mismos tópicos a través de la red DDS. La topología completa del sistema en su versión final integra percepción instantánea, conciencia espacial y mapeo local, tal como se muestra en la [Figure 4.3](#).

Esta organización responde a los condicionantes planteados en la introducción: concentra en la Raspberry Pi 5 el recorrido funcional necesario para producir la salida lógica, emplea representaciones compactas entre etapas y permite trasladar las herramientas opcionales de visualización y depuración a una computadora externa. Además, conserva la percepción frontal como ruta independiente de la odometría y del mapa local, de modo que la memoria espacial de v2 aporta información contextual sin sustituir la observación instantánea.

El [Anexo A](#) muestra la correspondencia entre la arquitectura, las decisiones metodológicas y el código fuente mediante un mapeo nodo $\rightarrow$ tópicos $\rightarrow$ archivos fuente.

4.2. Comparación de la topología entre versiones

Los diagramas de esta sección ofrecen una vista de conjunto de las tres configuraciones incrementales definidas en la [Sección 1.3.3](#). Su propósito es mostrar qué bloques se mantienen, se reemplazan o se agregan en cada versión; los tópicos, tipos de mensaje e interfaces de cada nodo se detallan posteriormente en la [Sección 4.4](#). La implementación conserva los modos `raw` y `filtered`, seleccionados mediante `pipeline_mode`, para ejecutar y comparar las configuraciones alcanzadas en v0 y v1.

4.2.1. Topología de v0: percepción directa

La versión inicial utiliza exclusivamente la percepción instantánea: imagen de profundidad de la cámara, conectada al pipeline mediante remapeos definidos en `nav_bringup`, reducción

a `DepthGrid` en `depth_grid_encoder`, fusión trivial (`passthrough`), visualización de la salida y serialización para su transmisión UART. La topología resultante se muestra en la Figura 4.1.

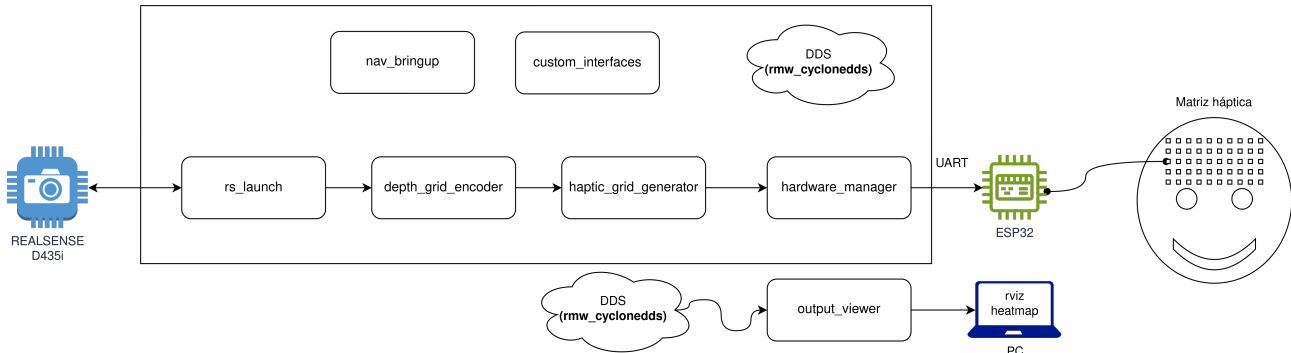


Figura 4.1: Arquitectura v0.

4.2.2. Topología de v1: filtrado geométrico

Sobre v0 se incorpora `depth_obstacle_filter` para suprimir suelo y techo, alineando la nube a gravedad y conservando en la salida únicamente obstáculos relevantes para la marcha. Esta etapa reemplaza la fuente directa de `DepthGrid`, pero conserva sin cambios los nodos de generación háptica y transmisión. La topología resultante se muestra en la Figura 4.2.

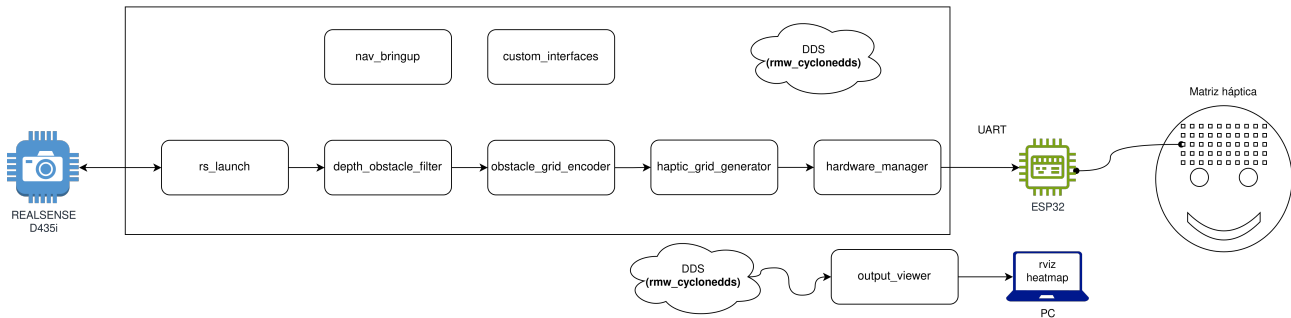


Figura 4.2: Arquitectura v1.

4.2.3. Topología de v2: memoria y riesgo contextual

La tercera versión agrega memoria espacial mediante `local_mapper` y una capa de riesgo contextual con `spatial_awareness`. Esta capa complementa la percepción frontal instantánea con alertas de ocupación lateral o trasera, aprovechando información acumulada en el mapa local y el estado de movimiento del usuario. La topología resultante se muestra en la Figura 4.3.

4.3. Infraestructura del sistema

El sistema utiliza paquetes de soporte para definir los mensajes, los parámetros y los modos de ejecución compartidos, componentes opcionales para observar su funcionamiento y una interfaz estable con el sensor. Los tópicos publicados por el driver RealSense se remapean a nombres internos, de modo que la configuración de la cámara puede cambiar sin modificar los nodos de procesamiento.

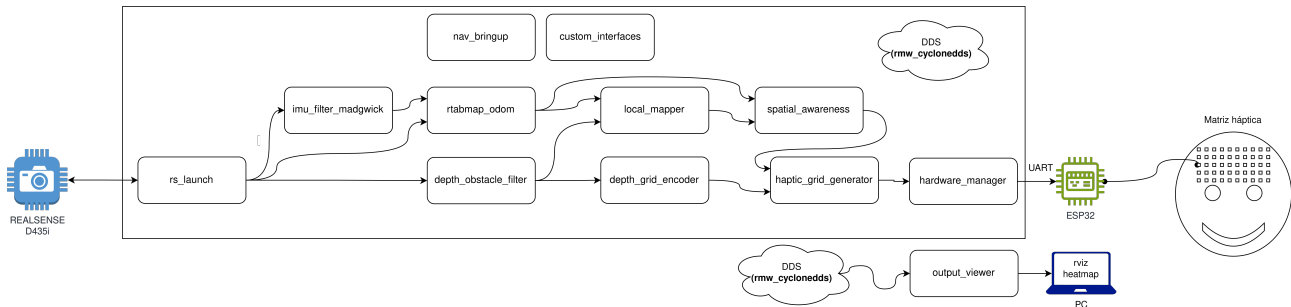


Figura 4.3: Arquitectura v2.

4.3.1. Paquetes de soporte e infraestructura

El paquete `custom_interfaces` concentra los tipos de mensajes específicos del sistema. En la cadena de percepción frontal, `DepthGrid` modela una grilla compacta de celdas y `DepthCellStats` encapsula la estadística de cada celda (media, mínimo, máximo y conteo de puntos válidos). En la cadena de actuación, `HapticGrid` representa la salida final de intensidades hápticas con dos arreglos explícitos: `intensities` y `active`. Para la conciencia espacial, `DirectionalRisk` describe el riesgo por dirección (activación, intensidad, distancia, velocidad de cierre y tiempo hasta colisión) y `SpatialAwareness` agrupa los tres canales `left`, `right` y `rear` junto con una bandera de validez global. Esta separación reduce el acoplamiento entre nodos y evita incorporar información de riesgo dentro de un mensaje destinado a percepción.

La configuración de lanzamiento se centraliza en `nav.launch.py`, perteneciente al paquete `nav_bringup`. Este archivo expone parámetros para habilitar o deshabilitar componentes y modos de ejecución, entre ellos: selección de pipeline de percepción (`raw/filtered/none`), activación de `local_mapper` y `spatial_awareness`, publicación en hardware y herramientas de visualización y depuración. Estas opciones determinan qué nodos se inician y con qué parámetros, mientras que la lógica de procesamiento permanece en los paquetes funcionales.

4.3.2. Visualización y diagnóstico

El paquete `output_viewer` agrupa componentes auxiliares desacoplados del procesamiento principal mediante tópicos ROS 2. Por ello pueden ejecutarse de forma opcional en el equipo de cómputo principal o en una PC conectada a la misma red DDS. El nodo `depth_grid_heatmap` representa la distribución espacial de intensidades y el estado de las celdas de `/perception/haptic_grid`, como matriz o como proyección del campo frontal, y puede incorporar los indicadores de riesgo lateral y posterior de `/spatial_awareness/collision_risk`. También ofrece un control opcional de la apertura angular del codificador; esta es la única interacción del paquete que puede modificar la configuración del flujo de percepción.

Las restantes capacidades cubren distintos niveles del diagnóstico. El nodo `grid_signal_monitor` supervisa continuidad, frecuencia y contenido de las señales de profundidad, nube de obstáculos y grilla. El nodo `odometry_path` transforma la odometría en una trayectoria acumulada, mientras que la configuración provista para RViz2 integra nubes intermedias, marcos de referencia, mapa, trayectoria e imagen de la cámara. Estas interfaces permiten observar el funcionamiento y localizar degradaciones entre etapas, pero no participan en la generación del comando háptico ni sustituyen su validación perceptual. Los detalles de implementación y operación se reservan para sus capítulos correspondientes; la trazabilidad de tópicos, tipos de

mensaje y componentes se presenta en el Anexo A.

4.3.3. Adquisición y adaptación de tópicos

El driver de la cámara, lanzado mediante `rs_launch.py` del paquete `realsense2_camera`, publica los flujos del sensor bajo el espacio de nombres `/camera/camera/`. En lo que sigue, para evitar repetición, ese prefijo se abrevia como `RS/`. En esta interfaz conviene distinguir entre nombre de tópico y tipo de mensaje: por ejemplo, `RS/depth/image_rect_raw`, `RS/color/image_raw` y `RS/aligned_depth_to_color/image_raw` son tópicos de imagen, todos transportados con tipo `sensor_msgs/Image`, mientras que `RS/depth/camera_info` y `RS/color/camera_info` son tópicos distintos que publican parámetros intrínsecos con tipo `sensor_msgs/CameraInfo`. De forma análoga, las mediciones inerciales llegan por tópicos específicos del driver y utilizan `sensor_msgs/Imu` como tipo de mensaje.

Cada nodo se suscribe mediante un nombre local, independiente del driver (por ejemplo, `depth/image`, `accel` o `rgb/image`), y los archivos de lanzamiento remapean ese nombre al tópico efectivo de RealSense. De esta forma, la selección de la fuente queda fuera de la lógica de procesamiento y no se crea una segunda copia de las imágenes. La Table 4.1 documenta los remapeos que componen las configuraciones integradas.

Nodo consumidor	Entrada del nodo	Tópico efectivo
<code>depth_to_matrix</code>	<code>depth/image</code>	<code>RS/depth/image_rect_raw</code>
<code>depth_obstacle_filter</code>	<code>depth/image</code>	<code>RS/depth/image_rect_raw</code>
	<code>depth/camera_info</code>	<code>RS/depth/camera_info</code>
	<code>accel</code>	<code>RS/accel/sample</code>
<code>imu_filter_madgwick</code>	<code>imu/data_raw</code>	<code>RS/imu</code>
<code>rtabmap_odom</code>	<code>rgb/image</code>	<code>RS/color/image_raw</code>
	<code>rgb/camera_info</code>	<code>RS/color/camera_info</code>
	<code>depth/image</code>	<code>RS/aligned_depth_to_color/image_raw</code>
	<code>imu</code>	<code>/imu/data_filtered</code>

Tabla 4.1: Entradas locales de cada nodo y tópicos efectivos seleccionados por los archivos de lanzamiento. `RS/` abrevia el prefijo `/camera/camera/`; `/imu/data_filtered` es la salida de `imu_filter_madgwick`, no un tópico del driver. El tópico `RS/imu` es la salida unificada que el controlador RealSense habilita para v2 mediante `unite_imu_method=1`.

Los remapeos de `depth_to_matrix`, `imu_filter_madgwick` y `rtabmap_odom` se declaran en `nav.launch.py`. Los de `depth_obstacle_filter` se declaran en su lanzador incluido, que lee de su archivo de parámetros los nombres RealSense usados como destino. Por tanto, el contrato desacoplado es la entrada local de cada nodo. Las herramientas de diagnóstico y grabación que necesitan observar el flujo global emplean los nombres del driver.

La separación de entradas inerciales sigue la evolución del sistema. En v1, `depth_obstacle_filter` consume solamente `RS/accel/sample` para estimar la vertical local; no utiliza el giróscopo ni Madgwick. En v2 se habilita además la IMU unificada `RS/imu`, formada a partir del acelerómetro y el giróscopo. Madgwick la convierte en `/imu/data_filtered`, que se entrega como entrada auxiliar a la odometría RGB-D de RTAB-Map.

4.4. Nodos y flujo de mensajes por versión

Esta sección describe las interfaces que materializan las configuraciones comparadas en la Sección 4.2: qué nodo produce cada representación, qué mensajes intercambia y qué componentes consumen la salida. Las tres versiones conservan un contrato de salida común, pero modifican progresivamente la fuente de la información espacial. V0 obtiene la grilla directamente de la profundidad; v1 la construye a partir de obstáculos filtrados y alineados con gravedad; v2 mantiene la ruta frontal de v1 y agrega memoria local y riesgo contextual.

4.4.1. Flujo de v0: profundidad y salida háptica

El nodo `depth_to_matrix` (paquete `depth_grid_encoder`) suscribe `depth/image`, remapeado desde `RS/depth/image_rect_raw` en `nav.launch.py`, y publica una representación compacta en `/perception/depth_grid` con tipo `custom_interfaces/msg/DepthGrid`. El nodo `haptic_grid_generator` consume esa interfaz y publica `/perception/haptic_grid` como `custom_interfaces/msg/HapticGrid`; finalmente, `hardware_manager` recibe la matriz de intensidades para transmitirla por UART. Este nodo no procesa distancias: adapta la salida lógica al rango configurado y la serializa para el microcontrolador. La frontera física del sistema se ubica en esa transmisión, ya que el receptor, la electrónica de potencia y los actuadores quedan fuera del alcance de la tesis. Esta versión establece la cadena funcional de referencia que conservan las versiones posteriores. El escalado, el formato de trama y el control de frecuencia se describen en el capítulo de Implementación.

La Figura 4.4 explicita el contrato de datos de esta ruta. Cada flecha indica el nombre del tópico, el tipo de mensaje y su contenido relevante. El diagrama distingue así la representación geométrica `DepthGrid`, la orden lógica `HapticGrid` y la interfaz de transmisión. La construcción de la grilla y la conversión a intensidad se desarrollan en las Secciones ?? y ??.

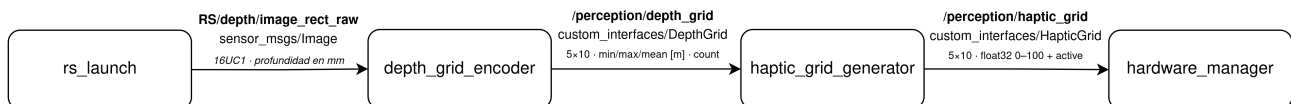


Figura 4.4: Mensajes funcionales de v0. La imagen de profundidad se reduce a `DepthGrid`, se transforma en una matriz lógica `HapticGrid` y se entrega a la interfaz de transmisión.

4.4.2. Flujo de v1: filtrado y codificación de obstáculos

El nodo `depth_obstacle_filter`, ejecutado mediante `depth_obstacle_filter_node` del paquete homónimo, suscribe la imagen de profundidad, sus parámetros intrínsecos y el acelerómetro del driver. El nodo publica `/depth_obstacle_filter/obstacle_cloud` como salida principal, y cuando se habilita la interfaz hacia mapeo local también publica `/depth_obstacle_filter/free_endpoints`. De forma adicional emite `/depth_obstacle_filter/camera_height` (`std_msgs/Float32`) para informar la altura estimada de cámara respecto del suelo. En TF publica la transformación dinámica `gravity_aligned_frame` → `camera_depth_optical_frame`. La nube de obstáculos es consumida por `obstacle_grid_encoder`, del paquete `depth_grid_encoder`, que la codifica angularmente y publica `/perception/depth_grid`. De este modo, v0 y v1 ofrecen el mismo contrato `DepthGrid` a las etapas posteriores, aunque lo construyen a partir de fuentes diferentes.

La Figura 4.5 muestra ese reemplazo sin presentar v1 como una cadena independiente. Las salidas funcionales hacia el codificador son la nube de obstáculos, expresada en el marco alineado

con gravedad, y la altura de cámara. El resultado vuelve a ser el mismo tipo `DepthGrid` de v0, por lo que la generación háptica y la interfaz de salida no cambian.

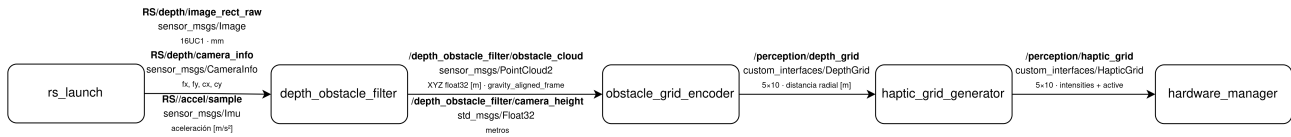


Figura 4.5: Mensajes funcionales de v1. La imagen, los intrínsecos y el acelerómetro permiten obtener una nube de obstáculos alineada y codificarla conservando el contrato `DepthGrid-HapticGrid` de v0.

La separación entre el filtrado geométrico de v1 y la rama de odometría de v2 evita que la percepción frontal dependa de la disponibilidad del mapa local. Los criterios de proyección, alineación, clasificación y codificación se describen en las Secciones ?? a ??.

4.4.3. Flujo de v2: odometría, mapeo y fusión contextual

La versión v2 mantiene en la rama inferior de la Figura 4.6 la percepción frontal de v1 y agrega dos recorridos auxiliares. El primero obtiene `nav_odom` mediante odometría RGB-D: RTAB-Map consume color, sus intrínsecos, imagen de profundidad registrada en el marco óptico de la cámara RGB y la orientación publicada por `imu_filter_madgwick`. El segundo combina esa odometría con las nubes de evidencia ocupada y libre para construir `OccupancyGrid`. A partir del mapa y del estado de movimiento, `spatial_awareness` publica el riesgo contextual que vuelve a `haptic_grid_generator`. Por lo tanto, la rama superior no reemplaza la percepción instantánea: aporta una segunda entrada que se fusiona antes de producir la única `HapticGrid` de salida.

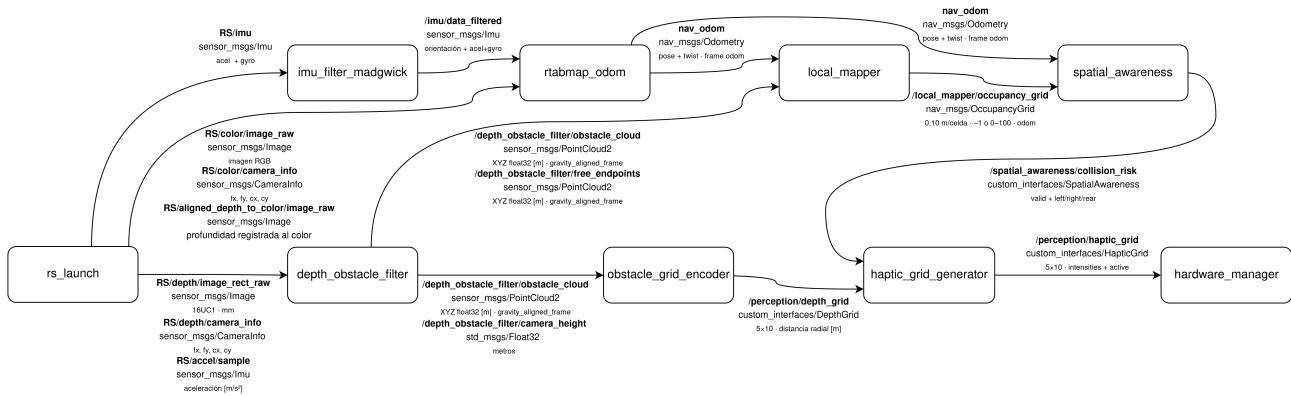


Figura 4.6: Mensajes funcionales de v2. La ruta frontal se complementa con odometría, memoria local y riesgo contextual, que se fusiona antes de publicar la matriz háptica final. Se muestran únicamente los mensajes que intervienen en el procesamiento funcional principal.

El nodo `local_mapper`, ejecutado mediante `occupancy_mapper_node` del paquete `local_mapper`, recibe las evidencias ocupada y libre publicadas por `depth_obstacle_filter`, junto con `nav_odom` y la altura de cámara. Es responsable de mantener la memoria espacial y publica `nav_msgs/OccupancyGrid` en `/local_mapper/occupancy_grid` con frame `odom`. También publica la TF dinámica `odom → gravity_aligned_frame` para mantener coherencia de marcos entre los

nodos de v2. El método de asociación temporal y actualización del mapa se desarrolla en las Secciones ?? y ??.

El nodo `spatial_awareness`, ejecutado mediante `spatial_awareness_node` del paquete homónimo, suscribe la grilla local, la odometría y el estado de apertura angular. Su responsabilidad es producir advertencias contextuales para los sectores izquierdo, derecho y posterior. El resultado se publica en `/spatial_awareness/collision_risk` con tipo `custom_interfaces/msg/SpatialAwareness`, que contiene un canal `DirectionalRisk` por sector y un indicador `valid` para señalar vigencia de sus entradas. El nodo `haptic_grid_generator`, cuyo ejecutable es `haptic_grid_generator_node`, recibe este mensaje junto con `/perception/depth_grid` y publica una única salida consolidada en `/perception/haptic_grid` con tipo `custom_interfaces/msg/HapticGrid`. Los criterios de evaluación del riesgo y de fusión se desarrollan en las Secciones ?? y ??.

Capítulo 5

Diseño del método (pipeline de percepción)

Capítulo 6

Implementación en ROS 2

Capítulo 7

Metodología experimental

Capítulo 8

Resultados

Capítulo 9

Discusión

Capítulo 10

Conclusiones

Capítulo 11

Trabajo futuro

Apéndice A

Trazabilidad de nodos y código fuente

A continuación se exponen tres tablas de referencia que relacionan los nodos principales del sistema, sus paquetes y los tópicos ROS 2 relevantes de entrada y salida. Esta trazabilidad facilita identificar, para cada bloque funcional descrito en los capítulos de Arquitectura, Método e Implementación, qué información consume y qué información publica. Las Tablas [A.1](#), [A.2](#) y [A.3](#) corresponden a la configuración v2 completa del sistema, esto es, *pipeline filtered*, odometría `rtabmap_odom` y activación de `spatial_awareness`.

Nodo	Paquete	In	Out
camera (realsense2_camera_node)	realsense2_camera	hardware	/camera/camera/imu (unificada para v2), /camera/camera/color/image_raw, /camera/camera/color/camera_info, /camera/camera/aligned_depth_to_color/image_raw, /camera/camera/depth/image_rect_raw, /camera/camera/depth/camera_info
imu_filter_madgwick	imu_filter_madgwick	/camera/camera/imu	/imu/data_filtered
rgbd_odometry	rtabmap_odom	/camera/camera/color/image_raw, /camera/camera/color/camera_info, /camera/camera/aligned_depth_to_color/image_raw, /imu/data_filtered	nav_odom
depth_obstacle_filter	depth_obstacle_filter	/camera/camera/depth/image_rect_raw, /camera/camera/depth/camera_info, /camera/camera/accel/sample	/depth_obstacle_filter/obstacle_cloud, /depth_obstacle_filter/free_endpoints, /depth_obstacle_filter/camera_height, /depth_obstacle_filter/debug/*

Tabla A.1: Trazabilidad de adquisición, prefiltrado IMU y percepción geométrica.

Nodo	Paquete	In	Out
local_mapper	local_mapper	/depth_obstacle_filter/obstacle_cloud, /depth_obstacle_filter/free_endpoints, nav_odom, /depth_obstacle_filter/camera_height	/local_mapper/occupancy_grid, TF odom → gravity_aligned_frame
obstacle_grid_encoder	depth_grid_encoder	/depth_obstacle_filter/obstacle_cloud, /depth_obstacle_filter/camera_height	/perception/depth_grid, /perception/depth_grid/aperture_state_deg
spatial_awareness	spatial_awareness	/local_mapper/occupancy_grid, nav_odom, /perception/depth_grid/aperture_state_deg	/spatial_awareness/collision_risk, /spatial_awareness/debug/markers

Tabla A.2: Trazabilidad de mapeo local, codificación de grilla y conciencia espacial.

Nodo	Paquete	In	Out
haptic_grid_generator	haptic_grid_generator	/perception/depth_grid, awareness/collision_risk	/spatial_awareness /perception/haptic_grid
hardware_manager (grid.to.motor)	hardware_manager	/perception/haptic_grid	Trama UART
depth_grid_heatmap	output_viewer	/perception/haptic_grid, awareness/collision_risk	/spatial_awareness visualización
nav_bringup (launch)	nav_bringup	argumentos de ejecución y configuración general	orquestración del sistema completo

Tabla A.3: Trazabilidad de fusión háptica, salida, visualización y orquestación.

Bibliografía

- [1] S. D. Novich y D. M. Eagleman, «Using space and time to encode vibrotactile information: toward an estimate of the skin's achievable throughput,» *Experimental Brain Research*, vol. 233, n.º 10, págs. 2777-2788, 2015. DOI: [10.1007/s00221-015-4346-1](https://doi.org/10.1007/s00221-015-4346-1). dirección: https://eagleman.com/papers/novich_eagleman_2015.pdf.
- [2] V. Santos, R. B. Medeiros, M. T. Rocha y col., «Wearable Haptic Device as Mobility Aid for Blind People: Electronic Cane - Wearable Device for Mobility of Blind People,» *ResearchGate*, 2023. dirección: <https://www.researchgate.net/profile/Vagner-Dos-Santos-3/publication/367149471-Wearable-Haptic-Device-as-Mobility-Aid-for-Blind-People-Electronic-Cane-Wearable-Device-for-Mobility-of-Blind-People/links/63ceccf66fe15d6a57426662/Wearable-Haptic-Device-as-Mobility-Aid-for-Blind-People-Electronic-Cane-Wearable-Device-for-Mobility-of-Blind-People.pdf>.
- [3] A. L. Petsiuk y J. M. Pearce, «Low-Cost Open Source Ultrasound-Sensing Based Navigational Support for the Visually Impaired,» *Sensors*, vol. 19, n.º 17, pág. 3783, 2019. DOI: [10.3390/s19173783](https://doi.org/10.3390/s19173783). dirección: <https://www.mdpi.com/1424-8220/19/17/3783>.
- [4] I. RealSenseTM, *D400 Series Product Family - Datasheet*, [download-datasheet](#), 2019.
- [5] P. Bach-y-Rita, C. C. Collins, F. A. Saunders, B. White y L. Scadden, «Vision Substitution by Tactile Image Projection,» *Nature*, vol. 221, n.º 5184, págs. 963-964, 1969. DOI: [10.1038/221963a0](https://doi.org/10.1038/221963a0).
- [6] Y. H. Lee y G. Medioni, «RGB-D Camera Based Wearable Navigation System for the Visually Impaired,» *Computer Vision and Image Understanding*, vol. 149, págs. 3-20, 2016. DOI: [10.1016/j.cviu.2016.03.019](https://doi.org/10.1016/j.cviu.2016.03.019).
- [7] H. Zhang, L. Jin y C. Ye, «An RGB-D Camera Based Visual Positioning System for Assistive Navigation by a Robotic Navigation Aid,» *IEEE/CAA Journal of Automatica Sinica*, vol. 8, n.º 8, págs. 1389-1400, 2021. DOI: [10.1109/JAS.2021.1004084](https://doi.org/10.1109/JAS.2021.1004084).
- [8] Seventh Sense OÜ, *SuperBrain 1: Advanced Assistive Technology for the Blind*, <https://7sense.ee/superbrain-1/>, Accessed: 2026, 2024.
- [9] R. P. Foundation, *Raspberry Pi Documentation*, Accessed: 2024-10-04, 2012. dirección: <https://www.raspberrypi.org/documentation/>.
- [10] Open Source Robotics Foundation, *ROS: Robot Operating System*, <https://www.ros.org/>, Accessed: 2024-10-04, 2007.
- [11] A. Elfes, «Using Occupancy Grids for Mobile Robot Perception and Navigation,» *Computer*, vol. 22, n.º 6, págs. 46-57, 1989. DOI: [10.1109/2.30720](https://doi.org/10.1109/2.30720).
- [12] S. Thrun, W. Burgard y D. Fox, *Probabilistic Robotics*. MIT Press, 2005, ISBN: 978-0-262-20162-9.

- [13] D. Dakopoulos y N. G. Bourbakis, «Wearable Obstacle Avoidance Electronic Travel Aids for Blind: A Survey,» *IEEE Transactions on Systems, Man, and Cybernetics, Part C: Applications and Reviews*, vol. 40, n.º 1, págs. 25-35, 2010. DOI: [10.1109/TSMCC.2009.2021255](https://doi.org/10.1109/TSMCC.2009.2021255).
- [14] K. Yang, K. Wang, W. Hu y J. Bai, «Expanding the Detection of Traversable Area with RealSense for the Visually Impaired,» *Sensors*, vol. 16, n.º 11, pág. 1954, 2016. DOI: [10.3390/s16111954](https://doi.org/10.3390/s16111954).
- [15] O. Brock y O. Khatib, «High-Speed Navigation Using the Global Dynamic Window Approach,» págs. 341-346, 1999. DOI: [10.1109/ROBOT.1999.770002](https://doi.org/10.1109/ROBOT.1999.770002). dirección: https://khatib.stanford.edu/publications/pdfs/Brock_1999_ICRA.pdf.
- [16] J. E. Bresenham, «Algorithm for Computer Control of a Digital Plotter,» *IBM Systems Journal*, vol. 4, n.º 1, págs. 25-30, 1965. DOI: [10.1147/sj.41.0025](https://doi.org/10.1147/sj.41.0025).
- [17] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss y W. Burgard, «OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees,» en *Autonomous Robots*, vol. 34, 2013, págs. 189-206. DOI: [10.1007/s10514-012-9321-0](https://doi.org/10.1007/s10514-012-9321-0). dirección: <https://octomap.github.io>.
- [18] S. Macenski, F. Martin, R. White y J. G. Clavero, «The Marathon 2: A Navigation System,» *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, págs. 2718-2725, 2020. DOI: [10.1109/IROS45743.2020.9341207](https://doi.org/10.1109/IROS45743.2020.9341207).
- [19] O. J. Woodman, «An introduction to inertial navigation,» University of Cambridge, Computer Laboratory, inf. téc. UCAM-CL-TR-696, 2007. dirección: <https://www.cl.cam.ac.uk/techreports/UCAM-CL-TR-696.pdf>.
- [20] C. Forster, L. Carlone, F. Dellaert y D. Scaramuzza, «On-Manifold Preintegration for Real-Time Visual-Inertial Odometry,» *IEEE Transactions on Robotics*, vol. 33, n.º 1, págs. 1-21, 2017. DOI: [10.1109/TR0.2016.2597321](https://doi.org/10.1109/TR0.2016.2597321).
- [21] A. I. Mourikis y S. I. Roumeliotis, «A Multi-State Constraint Kalman Filter for Vision-aided Inertial Navigation,» en *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2007, págs. 3565-3572. DOI: [10.1109/ROBOT.2007.364024](https://doi.org/10.1109/ROBOT.2007.364024).
- [22] T. Qin, P. Li y S. Shen, «VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator,» *IEEE Transactions on Robotics*, vol. 34, n.º 4, págs. 1004-1020, 2018. DOI: [10.1109/TR0.2018.2853729](https://doi.org/10.1109/TR0.2018.2853729).
- [23] M. Labbé y F. Michaud, «RTAB-Map as an Open-Source Lidar and Visual SLAM Library for Large-Scale and Long-Term Online Operation,» *Journal of Field Robotics*, vol. 36, n.º 2, págs. 416-446, 2019. DOI: [10.1002/rob.21831](https://doi.org/10.1002/rob.21831).
- [24] S. O. H. Madgwick, A. J. L. Harrison y R. Vaidyanathan, «Estimation of IMU and MARG orientation using a gradient descent algorithm,» *2011 IEEE International Conference on Rehabilitation Robotics*, págs. 1-7, 2011. DOI: [10.1109/ICORR.2011.5975346](https://doi.org/10.1109/ICORR.2011.5975346).
- [25] Intel Corporation, *Tuning Intel RealSense Depth Cameras for Best Performance*, <https://dev.realsenseai.com/docs/tuning-depth-cameras-for-best-performance>, Accessed: 2026, 2020.
- [26] R. Hartley y A. Zisserman, *Multiple View Geometry in Computer Vision*, 2.^a ed. Cambridge University Press, 2004, ISBN: 978-0-521-54051-3.

- [27] D. Scharstein y R. Szeliski, «A Taxonomy and Evaluation of Dense Two-Frame Stereo Correspondence Algorithms,» *International Journal of Computer Vision*, vol. 47, n.º 1-3, págs. 7-42, 2002. DOI: [10.1023/A:1014573219977](https://doi.org/10.1023/A:1014573219977). dirección: <https://link.springer.com/article/10.1023/A:1014573219977>.
- [28] Open Robotics, *Interfaces: Topics, Services and Actions*, <https://docs.ros.org/en/humble/Concepts/Basic/Interfaces-Topics-Services-Actions.html>, ROS 2 Humble documentation. Accessed: 2026-08-11, 2026.
- [29] Open Robotics, *Internal ROS 2 Interfaces*, <https://docs.ros.org/en/humble/Concepts/About-Internal-Interfaces.html>, ROS 2 Humble documentation. Accessed: 2026-08-11, 2026.
- [30] Open Robotics, *Quality of Service Settings*, <https://docs.ros.org/en/humble/Concepts/Intermediate/About-Quality-of-Service-Settings.html>, ROS 2 Humble documentation. Accessed: 2026-08-11, 2026.
- [31] Open Robotics, *rosvbag2: Recording and Playback of ROS 2 Communications*, <https://github.com/ros2/rosvbag2>, Accessed: 2026-08-11, 2026.
- [32] R. C. Gonzalez y R. E. Woods, *Digital Image Processing*, 4th. Pearson, 2018, ISBN: 978-0133356724.
- [33] SBS Implementers Forum, «System Management Bus (SMBus) Specification, Version 2.0,» SBS Implementers Forum, inf. téc., 2000, Sección 4.1.1: Packet Error Code (CRC-8). dirección: <http://smbus.org/specs/smbus20.pdf>.
- [34] H. B. Menz, S. R. Lord y R. C. Fitzpatrick, «Acceleration patterns of the head and pelvis when walking on level and irregular surfaces,» *Gait & Posture*, vol. 18, n.º 1, págs. 35-46, 2003. DOI: [10.1016/s0966-6362\(02\)00159-5](https://doi.org/10.1016/s0966-6362(02)00159-5).
- [35] A. V. Oppenheim y R. W. Schaffer, *Discrete-Time Signal Processing*, 2.ª ed. Prentice Hall, 1999, ISBN: 0-13-754920-2.
- [36] S. O. H. Madgwick, «An Efficient Orientation Filter for Inertial and Inertial/Magnetic Sensor Arrays,» Internal report, 30 de abr. de 2010. dirección: https://x-io.co.uk/downloads/madgwick_internal_report.pdf (visitado 05-08-2026).
- [37] ROS Enhancement Proposals, *REP-103: Standard Units of Measure and Coordinate Conventions*, <https://www.ros.org/reps/rep-0103.html>, Accessed: 2026, 2010.